國立陽明交通大學
電機資訊國際學位學程
碩士論文

EECS International Graduate Program
National Yang Ming Chiao Tung University
Master Thesis

於網路靶場中基於 RL-LLM 的多世代共演化紅隊與藍隊 AI 代理

RL-LLM Based Co-evolving Multi-generational Red-team and Blue-team AI Agents in a Cyber Range

研 究 生：安尼克（Aniket Aggarwal）

指導教授：林盈達（Ying-Dar Lin）

中華民國一一五年六月

June 2026

於網路靶場中基於 RL-LLM 的多世代共演化紅隊與藍隊 AI 代理

RL-LLM Based Co-evolving Multi-generational Red-team and Blue-team AI Agents in a Cyber Range

研 究 生：安尼克　　Student: Aniket Aggarwal

指導教授：林盈達 博士　　Advisor: Dr. Ying-Dar Lin

國立陽明交通大學

電機資訊國際學位學程

碩士論文

A Thesis

Submitted to EECS International Graduate Program

College of Electrical Engineering and Computer Science

National Yang Ming Chiao Tung University

in Partial Fulfilment of the Requirements

for the Degree of

Master in

Electrical Engineering and Computer Science

June 2026

Taiwan, Republic of China

中華民國 一一五年六月

於網路靶場中基於 RL-LLM 的多世代共演化紅隊與藍隊 AI 代理

學生：安尼克　　指導教授：林盈達 博士

國立陽明交通大學 電機資訊國際學位學程

摘　　要

有效的網路防禦需要藍隊智慧代理能夠持續適應日益複雜的紅隊對手。現有的人工智慧驅動方法通常將其中一方針對固定對手進行訓練，或允許紅隊與藍隊智慧代理同時更新其策略。然而，固定對手訓練可能導致策略過度擬合，而同步訓練則可能引入非平穩性所造成的不穩定問題。爲了解決此問題，本論文提出一個基於 RL-LLM 的多世代共同演化框架，並使用交替式共同演化方法在網路靶場中訓練紅隊與藍隊人工智慧代理。在交替式共同演化中，首先固定藍隊代理，並訓練紅隊代理以繞過其防禦；接著，固定改進後的紅隊代理，並訓練藍隊代理以偵測並抑制其攻擊。紅隊代理使用強化學習產生跨越 MITRE ATT&CK 戰術的多步驟攻擊鏈，而藍隊代理則使用強化學習從網路日誌中偵測惡意活動並選擇防禦回應。實驗結果顯示，面對演化後的紅隊代理，交替式共同演化方法展現出逐步提升的效果，其中藍隊抑制率由第 0 代的 4% 提升至第 3 代的 95.1%。同時，紅隊代理也持續演化，並在連續世代中發現了 533 條獨特的攻擊鏈。研究結果顯示，隨著藍隊效能提升，成功的紅隊攻擊變得更加困難，且需要更長、更深入的攻擊鏈序列，表示交替式共同演化方法不僅提升了防禦效果，同時也迫使紅隊代理持續演化，而非停滯不前。

關鍵字：紅藍共同演化、強化學習、大型語言模型、MITRE ATT&CK

RL-LLM Based Co-evolving Multi-generational Red-team and Blue-team AI Agents in a Cyber Range

Student: Aniket Aggarwal Advisor: Dr. Ying-Dar Lin

EECS International Graduate Program
National Yang Ming Chiao Tung University

Abstract

Effective cyber defense requires Blue-team agents that can continuously adapt to increasingly sophisticated Red-team adversaries. Existing AI-driven methods usually train one side against a fixed opponent or allow Red-team and Blue-team agents to update their policies simultaneously. However, fixed-opponent training can cause policy overfitting, while simultaneous training may introduce non-stationary instability. To address this problem, this thesis proposes an RL-LLM-based Multi-generational Co-evolution (MG-CoE) framework for training Red-team and Blue-team AI agents in a cyber range using Alternating Co-Evolution (ACE). In ACE, the Blue-team agent is first fixed while the Red-team agent trains to bypass its defenses; then, the improved Red-team agent is fixed while the Blue-team agent trains to detect and suppress its attacks. The Red-team agent uses RL to generate multi-step attack chains across MITRE ATT&CK tactics, while the Blue-team agent uses RL to detect malicious activity and select defensive responses using logs. The results show that against an evolved Red-team agent, ACE demonstrates progressive improvement, with Blue-team suppression increasing from 4% at Generation 0 to 95.1% at Generation 3. Meanwhile, the Red-team agent continued to evolve, discovering 533 unique kill chains over successive generations. The findings suggest that as Blue-team performance improved, successful Red-team attacks became harder to achieve and required longer and deeper kill-chain sequences, indicating that ACE improved defensive effectiveness while still forcing the Red-team agent to evolve rather than stagnate.

Keywords: Red-Blue Co-evolution, Reinforcement Learning, Large Language Models, MITRE ATT&CK

Acknowledgement

回顧這段人生旅程，我滿懷感激之情。這篇論文的完成從來不是一條輕鬆的道路，途中充滿了疑惑、不眠之夜，以及幾度幾乎想要放棄的時刻。然而，在這些困難的時刻推動我繼續前行的，是他人的支持與善意。

首先，我要向我的指導教授林盈達老師表達我最深的感謝。您對「爲什麼」的不斷追問，不僅塑造了我研究的邏輯，更深刻地影響了我面對問題的思考方式。您教會我如何更精確、更有耐心地探索問題，並保有好奇心。這些學習將超越論文本身，伴隨我未來的道路。我也誠摯感謝賴源正老師與黃仁竑老師，您們在定期討論中所提供的指導與建議，讓我不斷改進研究內容，並體會到學術清晰與嚴謹的重要性。我特別感謝我的中文老師林佳慧教授，您不僅是出色的教師，更是我眞摯的朋友與個人靈感的來源。您溫暖的教學風格與不吝鼓勵讓每堂課都意義非凡。謝謝您一直相信我。

感謝 701 MIRC 實驗室的所有夥伴們——您們不僅是同事，更是陪我一起走過這段旅程的重要存在。我要特別感謝學長。您們的指導、見解與正能量對我意義非凡。也謝謝所有的室友與學弟妹們，感謝你們的歡笑、討論，以及那份共同堅持的心。這些回憶我將永遠珍藏於心中。

我也要感謝我的家人。即使相隔千里，您們每週的來電與不曾中斷的支持，始終是我最堅實的後盾。每一句鼓勵的話語都在我最困難的時候響起，給予我力量。謝謝我親愛的女友淳云，感謝你無盡的愛、理解與耐心。你總是在我低落時鼓勵我，提醒我自己的力量，並陪我慶祝每一個小小的成就。在我風雨飄搖之中，你的存在總能讓我平靜下來。感謝分布在世界各地的摯友們，我非常想念你們。即使我們相隔遙遠，你們的支持依然帶給我巨大的力量。謝謝你們讓我知道，我從不孤單。

最後，我要感謝自己。這是一段充滿挑戰的旅程，無論是學術、情緒或生活。我曾在實驗室與房間裡熬過無數個夜晚，靠著咖啡與那份默默的堅持撐過來。尤其在第一年，有過無數次想要放棄的時候，但我沒有放棄，我堅持下來，並完成了它。

這篇論文不僅是研究成果的體現，更是人與人之間情感、成長與韌性的結晶。我將你們的名字一一珍藏於此篇頁之中。

Table of Contents

摘要 i

Abstract ii

Acknowledgement iii

Table of Contents iv

List of Figures vii

List of Tables viii

1 Introduction 1

2 Background and Related Work 4

2.1 Cybersecurity Training 4

2.2 MITRE ATT&CK Framework 5

2.3 Reinforcement Learning and Parameterized Cyber Actions 5

2.4 Related Work 7

3 Problems: Kill Chain Completion and Disruption 9

3.1 Notations 9

3.2 An Illustrative Example 11

3.3 Problem Statements 12

4 Multi-generational Co-Evolution Framework 14

4.1 Solution Overview 14

4.2 Cyber Range 16

4.3 Red-team Agent 18

4.3.1 Red Action Space 18

4.3.2 Technique Schema 19

4.3.3 Action Selection and Validation 20

4.3.4 Red-team Reward Design 22

4.4 Blue-team Agent 23

4.4.1 Detection Component 24

4.4.2 Blue-team Action Space . . . 24
4.4.3 Response Component . . . 25
4.4.4 Blue-team Reward Design . . . 26
4.5 MG-CoE Execution Loop . . . 27
4.6 Running Example . . . 29
4.6.1 Blue-team Success by Blocking the Target . . . 29
4.6.2 Red-team Success After an Insufficient Blue-team Response . . . 30
5 Implementation . . . 32
5.1 Implementation Overview . . . 32
5.2 Open-Source Tools and Libraries . . . 34
5.3 Cyber Range and Testbed Architecture . . . 35
5.4 Red-team Agent Implementation . . . 36
5.5 Blue-team Agent Implementation . . . 37
5.5.1 Detection . . . 37
5.5.2 Response . . . 38
6 Experiments and Evaluations . . . 39
6.1 Experimental Setup . . . 39
6.2 Alternating Co-Evolution vs FOT vs Simultaneous Co-Evolution . . . 41
6.2.1 Blue-team Suppression Against Moderate and Strong Red-team Agents 41
6.2.2 Training Reward Stability . . . 42
6.2.3 Training Episode Efficiency . . . 43
6.2.4 Kill-Chain Length Behavior . . . 44
6.3 ACE Generational Dynamics . . . 45
6.3.1 Blue-team Suppression Across Generations . . . 45
6.3.2 Red-team Kill-Chain Diversity . . . 46
6.3.3 Technique Rotation Across Generations . . . 47
6.3.4 Kill-Chain Depth Analysis . . . 48
6.3.5 Cross-Generation Generalization . . . 49

7 Conclusion and Future Work 50
7.1 Conclusion 50
7.2 Future Work 51
References 52

List of Figures

2.1 MITRE ATT&CK For Enterprise 5

2.2 Reinforcement learning interaction loop. 6

3.1 Red-team and Blue-team decision problem 12

4.1 Multi-generation alternating training pattern. 15

4.2 Step-level Red-team–Blue-team loop. 16

4.3 Technique Schema Example. 20

4.4 Red-team agent action-selection workflow. 21

4.5 Blue-team agent detection and response workflow. 25

4.6 Example episodes. 29

5.1 System architecture. 33

5.2 Red-team and Blue-team network architectures. 37

6.1 Blue-team suppression rate against Red-team Gen 1 and Gen 3. 41

6.2 Moving-average Red-team reward and Blue-team reward across training progress. 42

6.3 Training episode cost. 43

6.4 Kill-chain length distributions across Blue-team series 44

6.5 ACE Blue-team suppression against Red-team Gen 1 and Red-team Gen 3. . . 45

6.6 New and cumulative unique successful kill chains across Red-team generations. 46

6.7 ACE technique frequency heatmap across generations. 47

6.8 ACE kill-chain progression depth distribution. 48

6.9 ACE cross-generation Blue-team suppression rate heatmap. 49

List of Tables

2.1 Related Work on AI-Based Red-team–Blue-team. 7

3.1 Notation Table 10

4.1 Red-team Action Space. 19

4.2 Red-team Reward Design. 23

4.3 Blue-team Action Space. 25

4.4 Blue-team per-action operational costs. 27

4.5 Blue-team Response Rewards. 27

5.1 Modules and Responsibilities. 33

5.2 Tools and libraries. 34

Chapter 1

Introduction

Cyber attacks are increasing in frequency and complexity, requiring defenders to detect and respond quickly to evolving threats. To evaluate and improve such defensive capabilities, Red-team and Blue-team exercises rely heavily on human experts to simulate attackers and defenders. While effective, these exercises are costly, slow, and difficult to repeat consistently. As a result, scalable and repeatable training mechanisms are required to continuously evaluate and improve defensive capabilities.

Recent research has explored automated approaches for both offensive and defensive cybersecurity tasks. On the attacker side, systems have evolved from scripted attacks using tools such as Metasploit and Atomic Red Team [1, 2] toward reinforcement learning (RL)-based attack planning [3] and large language model (LLM)-driven techniques that generate adaptive payloads and evasion strategies [4]. On the defender side, automated systems rely on rule-based intrusion detection systems (IDS), endpoint detection and response (EDR) tools, and security information and event management (SIEM) correlation engines, which are effective for known patterns but struggle with novel threats [4–6]. Machine learning approaches detect anomalies in system and network behavior but often produce noisy results and lack direct mapping to response actions [7].

Several Cyber Range training platforms have been proposed to support such automated learning systems. CyGIL provides a simulated network environment where RL agents learn attack or defense policies [7]. Cyberwheel introduces a Cyber Range RL framework where autonomous agents interact in attack-defense loops [6]. However, most existing systems train agents against a fixed opponent, where one agent remains unchanged while the other learns. This fixed-opponent training provides stability but leads to overfitting: because the learning agent optimizes its policy exclusively against one static adversary, it never encounters varied or evolving strategies and therefore fails to generalize to stronger or unseen opponents.

To allow both agents to interact, some approaches employ simultaneous co-evolution, where

both Red-team and Blue-team agents update their policies at the same time [8]. Although this introduces mutual adaptation, it often results in instability, oscillations, and difficulty in determining which agent is actually improving, because both policies shift simultaneously.

More generally, these limitations arise because existing methods lack a generation-based mechanism. Without organizing training into clearly defined generations, where each agent must demonstrate measurable improvement over its predecessor before advancing, it becomes impossible to verify whether a well-performing final agent is genuinely stronger or has simply converged against a particular opponent by chance. Even in interactive settings where both agents adapt, the absence of a generation-based mechanism means that the trajectory of improvement remains opaque and cannot be systematically evaluated.

Therefore, the core problem addressed in this thesis is to design a training framework that enables stable co-evolution process across multiple generations, allowing both attacker and defender agents to continuously improve without stagnation, while ensuring that improvement is verifiable.

To address this problem, this thesis proposes RL-LLM based **Multi-generational Co-Evolution (MG-CoE) framework** in which training is organized through an alternating co-evolution process. In each generation, one agent is held fixed at its currently trained policy while the opposing agent trains against it. Once the training agent demonstrates sufficient improvement over its predecessor, it is promoted to the next generation. The roles are then switched: the newly promoted agent becomes the fixed opponent, and the other agent trains against it. This alternating structure creates a controlled progression in which each agent faces a stronger but stable opponent. Because a promotion criterion must be satisfied before either agent advances, improvement at every generation is measurable and verifiable rather than assumed.

In our design, both agents operate within a simulation environment. The Red-team agent uses RL to select MITRE ATT&CK techniques and associated parameters, forming multi-stage attack kill chains with support from the LLM. The Blue-team agent observes logs from the Cyber Range, classifies evidence using a learned detection component, and applies RL to select a proportionate defensive response action. RL is used because cyber attack and defense are sequential decision-making problems, where actions taken at early stages affect long-term outcomes [3].

Specifically, this thesis uses Double Deep Q-Networks (DDQN) because it can produce more stable value targets across long multi-step training episodes [9].

The contributions of this thesis are threefold. From a problem perspective, it addresses stagnation and instability in automated cyber defense training by introducing a multi-generation structure with explicit promotion criteria. From a solution perspective, it proposes the MG-CoE framework, which uses DDQN-based learning in a unified Cyber Range system and introduces an alternating co-evolution training mechanism to improve Red-team and Blue-team agents across generations. From an evaluation perspective, it analyzes agent performance across generations and compares different training methods to demonstrate that Alternating Co-evolution produces more stable and measurable improvement than others.

The remainder of this thesis is organized as follows. Chapter 2 presents background and related work. Chapter 3 defines the problem formulation. Chapter 4 describes the proposed solution. Chapter 5 details the system implementation. Chapter 6 presents experimental evaluation and results. Chapter 7 concludes the thesis and discusses future work.

Chapter 2

Background and Related Work

This chapter first introduces the background required to understand the proposed multi-generational Red-Blue training framework. It first explains cybersecurity training. It then introduces MITRE ATT&CK as a structured framework for modeling attack lifecycles. Next, it reviews reinforcement learning for cyber decision-making, including parameterized actions, and then discusses related work on AI-Based Red-team and Blue-team agents.

2.1 Cybersecurity Training

Cybersecurity training is used to evaluate and improve an organization's ability to detect, analyze, and respond to cyber threats. A common form of such training is the Red-Blue team exercise. The Red Team emulates attacker behavior, while the Blue Team monitors the environment, detects suspicious activity, and applies defensive actions to reduce or stop the attack.

Red Team activities are usually performed as multi-step attack campaigns rather than isolated actions. An attacker may begin with reconnaissance, gain initial access, execute commands, escalate privileges, move laterally, and finally reach a high-value target. Each step may create the conditions needed for later steps. Therefore, cyber attacks are naturally sequential.

Blue Team activities also follow a sequential process. Defenders collect evidence from security tools such as IDS, EDR, firewalls, and SIEM systems [10]. They then analyze alerts, confirm malicious behavior, contain the threat, remove attacker artifacts, and recover affected systems. However, alerts are often incomplete, noisy, or ambiguous, making response selection difficult.

Manual Red-Blue exercises are useful but expensive, time-consuming, and difficult to repeat under identical conditions. Automated training environments address this limitation by allowing attack and defense agents to interact repeatedly in a controlled environment. This supports systematic evaluation of both attacker progression and defender response.

2.2 MITRE ATT&CK Framework

MITRE ATT&CK is a knowledge base of adversary behavior based on real-world cyber attacks [11]. It organizes attacker behavior into tactics and techniques. A tactic represents the attacker's high-level objective, such as Initial Access, Execution, Credential Access, Lateral Movement, or Impact. A technique represents the concrete method used to achieve that objective.

This structure is useful for attack lifecycle modeling. A realistic attack is not a single event, but a sequence of related techniques. For example, credential access may enable lateral movement, and privilege escalation may enable defense evasion or impact. Therefore, ATT&CK provides a structured way to represent multi-stage kill chains.

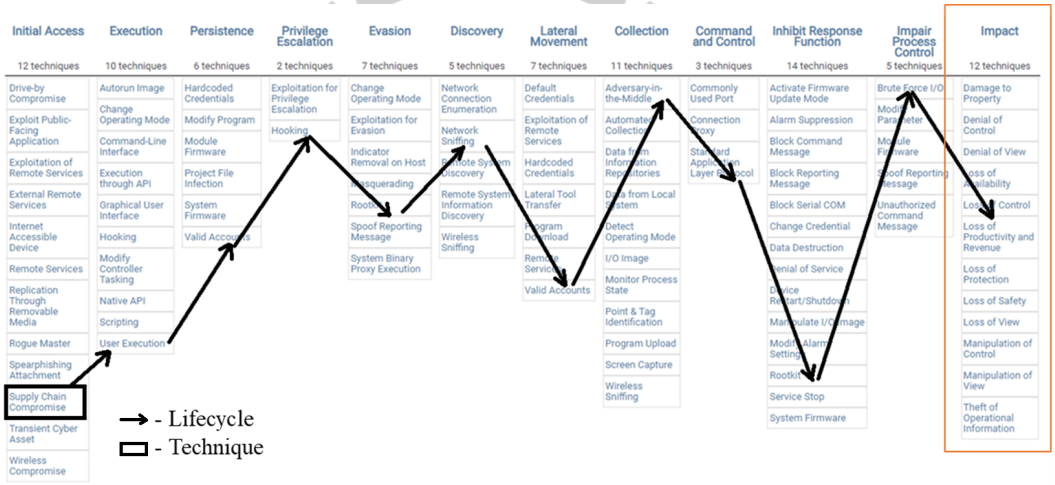

Figure 2.1: MITRE ATT&CK For Enterprise

Using ATT&CK also makes attack modeling more interpretable. Instead of describing attacks with arbitrary labels, each attack step can be mapped to a recognized tactic and technique. This helps compare attack behavior across different systems, datasets, and experiments.

2.3 Reinforcement Learning and Parameterized Cyber Actions

Reinforcement learning is a method for sequential decision-making [12]. An agent observes the current state, selects an action, receives a reward, and moves to a new state. Through repeated

interaction, the agent learns a policy that maximizes long-term cumulative reward [13].

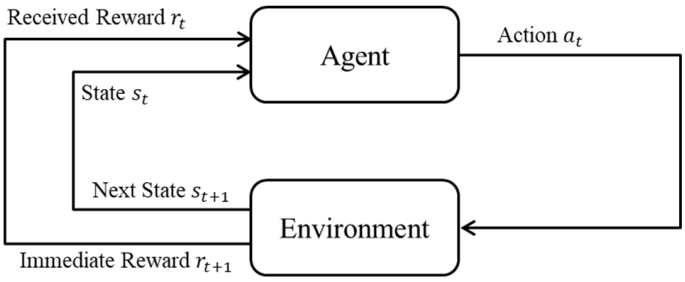

Figure 2.2: Reinforcement learning interaction loop.

Cyber attack and defense are suitable for reinforcement learning because both are sequential. On the attack side, early techniques can affect whether later techniques become possible. On the defense side, early response actions can stop, delay, or fail to affect attacker progression. Therefore, the value of an action depends not only on its immediate result but also on its long-term consequence.

A reinforcement learning problem is commonly modeled as a Markov Decision Process (MDP), which includes states, actions, transition dynamics, rewards, and a discount factor. In cybersecurity, the state may include host status, access level, privilege level, detected alerts, or attack stage. The action may be an attack technique or a defensive response. The reward measures whether the action improves the attacker's progress or the defender's protection.

Q-learning learns an action-value function $Q(s, a)$, which estimates the expected future reward of taking action a in state s. Deep Q-Networks (DQN) extend this idea by using a neural network to approximate Q-values in larger state spaces. Double Deep Q-Networks (DDQN) further improve DQN by reducing Q-value overestimation, making learning more stable in complex decision environments [9].

In cyber operations, actions are often not simple discrete choices. Many attack and defense actions require both an action type and concrete parameters. For example, selecting a remote execution technique is not enough; the agent must also choose a target host, port, credential, command, or payload. Similarly, a defensive response may require selecting the affected host, user account, process, IP address, or credential to act on.

This creates a parameterized action problem. A parameterized action contains a discrete action and a set of values required to execute it. In cybersecurity, this is more realistic than

treating each technique or response as a simple label, because the success of an action depends on whether its parameters are valid in the current environment.

Parameterized Action Markov Decision Processes (PAMDPs) provide a formal way to model this type of problem [14]. They are useful for cybersecurity because attack and defense actions often require concrete targets, credentials, processes, users, or network objects.

2.4 Related Work

Table 2.1: Related Work on AI-Based Red-team–Blue-team.

Study	Red Team AI	Blue Team AI	RL Algorithm		Training Method			Multi-Generation
			Red	Blue	Fixed Opponent	Sim. Co-Evo	Alt. Co-Evo	
[4]	Scripted	RL+LLM	–	PPO	✓	–	–	–
[5]	Scripted	RL-based	–	DQN, PPO	✓	–	–	–
[3]	RL-based	Scripted	PPO	–	✓	–	–	–
[15]	RL-based	Scripted	DQN variants	–	✓	–	–	–
[6]	Scripted	RL-based	–	DQN	✓	–	–	–
[8]	RL-based	RL-based	PPO, A2C	PPO, A2C	-	✓	–	–
MG-CoE(Ours)	RL + LLM	RL + LLM	DDQN	DDQN	–	–	✓	✓

Recent research on AI-driven cybersecurity has explored both RL and LLMs for automating Red-team and Blue-team behavior [13, 16]. Table 2.1 summarizes representative works according to their agent design, learning algorithm, and training method.The studies can be organized according to the training method used: Fixed-Opponent training, in which one agent trains against a permanently non-evolving opponent; Simultaneous Co-Evolution, in which Red-team and Blue-team agents are trained together at the same time; and Alternating Co-Evolution, in which agents are trained alternatively in multiple generations.

For example, Castro et al. [4], uses Fixed-Opponent training by pairing a scripted Red-team agent with an RL-LLM-based Blue-team defender trained using PPO. Their work demonstrates that LLMs can improve defensive reasoning against predefined attack sequences. Emerson et al. [5] similarly employ scripted Red-team adversaries while training RL-based Blue-team agents using PPO and DQN variants in controlled cyber-range environments, with a focus on training stability. On the offensive side, Nguyen et al. [3] and Kuppa et al. [15] develop RL-based Red-team agents for automated attack planning while keeping the Blue-team agent

scripted, emphasizing attacker-side generalization. Oesch et al. [6] train a DQN-based Blue-team agent against scripted Red-team opponents in a high-fidelity cyber-range environment, prioritizing the realism of the simulation infrastructure. As shown in Table 2.1, Fixed-Opponent training offers stable learning dynamics but is fundamentally limited by its reliance on a non-evolving adversary. As a result, the trained agent may not encounter sufficiently varied or adaptive strategies, limiting its ability to generalize beyond the fixed opponent.

On the other hand, Kunz et al. [8], which extends the Microsoft CyberBattleSim environment [17] by adding Blue-team training and introducing Simultaneous Co-Evolution. In their Simultaneous Co-Evolution RL-based Red-team and Blue-team agents are trained together using PPO and A2C which enables mutual adaptation but it introduces non-stationarity because both agents update their policies at the same time and each agent faces an opponent whose behavior changes during training, making convergence less stable and performance gains harder to attribute.

Across these training methods, a consistent gap is evident, Fixed-Opponent training provides stable learning but it does not create adaptive adversaries whereas Simultaneous Co-Evolution introduces non-stationarity, making training less stable. As a result, neither approach provides a structured mechanism for producing stable, and progressive improvement in both agents over multiple generations.

In contrast, the proposed *MG-CoE* framework combines DDQN-based learning with LLM-supported cyber reasoning in a structured multi-generation training process. Instead of training against only one static opponent or updating both agents at the same time, MG-CoE alternates the learning role across generations: one agent trains against a fixed opponent, must satisfy a promotion criterion, and then becomes the next stable opponent for the other side. This design preserves the stability of Fixed-Opponent training while still allowing both Red-team and Blue-team agents to evolve.

Chapter 3

Problems: Kill Chain Completion and Disruption

This chapter formalises the Cyber Range Red-team–Blue-team problem addressed in this thesis. The chapter is organised into three parts. First, Section 3.1 defines the notation used throughout the thesis. Second, Section 3.2 presents an illustrative example to ground the notation in a concrete Red-team–Blue-team encounter. Finally, Section 3.3 formalises the problem statement by specifying the inputs, outputs, objectives, and constraints of the Red-team and Blue-team agents within one episode.

3.1 Notations

Training operates across three nested levels, each with its own index symbol. A *generation* $n \in \{1, \ldots, N\}$ contains multiple training blocks in which one agent is held fixed at its current policy while the opposing agent trains against it. Generation 0 is a shared baseline in which both R_0 and B_0 train together from scratch and provides the starting point for further training.

Within generation n, the learning agent trains over a sequence of *episodes* indexed by $e \in \{1, \ldots, N_E\}$, where N_E is the maximum number of episodes in a training block. Each episode is one complete attacker–defender encounter that begins from the fixed base state $\mathcal{S}_{\text{base}}$ and ends when the Red agent reaches the goal host or the Blue agent applies hard containment.

Within episode e, the agents take *steps* indexed by j for the Red agent and i for the Blue agent. One step is one atomic interaction: the Red agent executes one technique and the Blue agent responds with one defensive action.

In short, generation n contains episodes $e = 1, \ldots, N_E$, and each episode contains step indices j (Red) and i (Blue). Every step-level symbol therefore carries the subscript pair (e, j) or (e, i), every episode-level symbol carries only e, and every generation-level symbol carries

only n. Table 3.1 summarises the notations.

Table 3.1: Notation Table

Category	Symbol	Description
Indices	$N \in \mathbb{N}$	Total number of generations.
	$n \in \{0, \ldots, N\}$	Generation index; $n = 0$ denotes the shared baseline.
	$N_E \in \mathbb{N}$	Maximum number of episodes per training block.
	$e \in \{1, \ldots, N_E\}$	Episode index within a training block.
	$j \in \mathbb{N}^+$	Red-team step index within an episode.
	$i \in \mathbb{N}^+$	Blue-team step index within an episode.
Cyber Range	$\mathcal{S}_{\text{base}}$	Initial Cyber Range state.
	$\mathcal{T}$	Catalogue of MITRE ATT&CK technique schemas.
	$\mathcal{K}$	Set of predefined kill-chain paths.
	$\kappa_e^R \in \mathcal{K}$	Red-team kill-chain path chosen at the start of episode e.
Agents	$\alpha \in \{R, B\}$	Agent identifier; R denotes the Red-team agent and B denotes the Blue-team agent.
	$R_n,\ B_n$	Red-team and Blue-team agents at generation n.
	S^α, with $\alpha \in \{R, B\}$	Action space of agent α.
	S^α_{succ}, with $\alpha \in \{R, B\}$	Successful outcome set of agent α; S^R_{succ} stores successful kill chains, while S^B_{succ} stores successful responses.
	η^α, with $\alpha \in \{R, B\}$	Promotion threshold of agent α.
	$y_e^\alpha \in \{0, 1\}$, with $\alpha \in \{R, B\}$	Episode-level outcome flag of agent α; $y_e^R = 1$ indicates Red-team success, while $y_e^B = 1$ indicates Blue-team success.
Red Step Variables	$O^R_{e,j}$	Red-team observation at step j of episode e.
	$t^R_{e,j} \in \mathcal{T}$	ATT&CK technique selected by the Red-team agent at step j.
	$x^R_{e,j}$	Parameters used with technique $t^R_{e,j}$.
	$a^R_{e,j} = (t^R_{e,j}, x^R_{e,j}) \in S^R$	Red-team action at step j of episode e.
	$P^R_{e,j} \in \mathbb{R}$	Red-team reward at step j of episode e.
	$\widetilde{R}_n$	Candidate Red-team agent being trained for generation n.
	ρ^R	Red-team success rate.
Blue Step Variables	$O^B_{e,i}$	Blue-team observation at step i of episode e.
	$F_{e,i}$	Feature vector encoded from the Blue-team observation $O^B_{e,i}$.
	$d_{e,i} \in \{0, 1\}$	Detection flag at step i, where $d_{e,i} = 1$ indicates that malicious activity is detected.
	$a^B_{e,i} \in S^B$	Blue-team defensive action at step i of episode e.
	$P^B_{e,i} \in \mathbb{R}$	Blue-team reward at step i of episode e.
	$\widetilde{B}_n$	Candidate Blue-team agent being trained for generation n.
	ρ^B	Blue-team suppression rate.

Two clarifications are essential. First, the kill-chain path κ_e^R is chosen at the start of episode e to guide the Red-team agent's tactic ordering. It is a path-level variable rather than a per-step variable, although the path may be extended within an episode as the campaign progresses. Second, the outcome flag y_e^α records whether agent α achieves its episode-level objective. For the Red-team agent, $y_e^R = 1$ indicates successful kill-chain completion and goal-host compromise.

For the Blue-team agent, $y_e^B = 1$ indicates successful kill-chain disruption before Red-team success. The success sets S_{succ}^R and S_{succ}^B collect these successful outcomes across episodes.

3.2 An Illustrative Example

To make the notation concrete, consider one episode in generation $n = 1$, indexed as episode e. The Red-team agent R_1 begins by selecting the kill-chain path $\kappa_e^R = \langle$Execution $\rightarrow$ Credential Access $\rightarrow$ Lateral Movement $\rightarrow$ Impact$\rangle$ from $\mathcal{K}$, and the Cyber Range is reset to $\mathcal{S}_{\text{base}}$.

At step j, suppose the active tactic in κ_e^R is Credential Access. The Red-team agent observes $O_{e,j}^R$ and selects a candidate technique $t_{e,j}^R =$ `T1003 LSASS Dump` and then resolves the parameters $x_{e,j}^R$. The resulting action $a_{e,j}^R = (t_{e,j}^R, x_{e,j}^R) \in S^R$ is passed to the Range Controller. The Range Controller executes the action in the Cyber Range, updates the Cyber Range state with the harvested credentials, and returns reward $P_{e,j}^R > 0$.

At the same step, the Blue-team agent B_1 receives observation $O_{e,i}^B$ containing Cyber Range telemetry. The detection component extracts the feature vector $F_{e,i}$ and sets the detection flag $d_{e,i} = 1$. The response component then selects $a_{e,i}^B =$ `revoke_credentials` from S^B. Because this response invalidates the harvested credentials and removes the Red-team agent's current credential advantage, the host status returns to Normal. Therefore, the Blue-team reward for this response is positive, $P_{e,i}^B > 0$, and the response is added to S_{succ}^B.

If the Red-team agent later reaches the goal host, the episode is marked as a Red-team success with $y_e^R = 1$ and added to S_{succ}^R. If the Blue-team agent disrupts the kill chain before the goal host is reached, the episode is marked as a Blue-team success with $y_e^B = 1$ and added to S_{succ}^B.

After the training block of N_E episodes, the Red-team candidate is evaluated against B_1. If its performance exceeds the promotion threshold η^R, it is promoted to R_2 and serves as the fixed opponent for the next Blue-team training block. If it does not satisfy the promotion threshold, the Red-team candidate continues training against the fixed Blue-team agent until the required improvement is achieved.

3.3 Problem Statements

The interaction between the Red-team agent and the Blue-team agent is modelled as a sequential adversarial decision-making problem inside the Cyber Range. In each episode, the Red-team agent attempts to complete a kill chain and reach a goal host, while the Blue-team agent attempts to detect, respond to, and disrupt the attack before the kill chain is completed. The episode terminates when the Red-team agent reaches the goal host, the Blue-team agent successfully disrupts the kill chain. Figure 3.1 illustrates the episode-level decision problem.

Figure 3.1: Red-team and Blue-team decision problem

Red-team Problem: Kill Chain Completion

- **Input:**
  - $O^R_{e,j}$: Red-team observation at step j of episode e.
  - $\mathcal{T}$: catalogue of attack technique schemas.
  - $\mathcal{K}$: set of kill-chain paths.
  - S^R: Red-team action space.
- **Output:**
  - $\{a^R_{e,j}\}_{j\geq 1}$: sequence of Red-team actions.
  - $y^R_e \in \{0, 1\}$: Red-team outcome flag, where $y^R_e = 1$ indicates that the Red-team agent reaches the goal host.
- **Objective:** maximise the Red-team episode-level success rate:

$$\max_{\widetilde{R}_n} \ \rho^R(\widetilde{R}_n \mid B_{n-1}), \quad \rho^R(\widetilde{R}_n \mid B_{n-1}) = \frac{1}{N_E} \sum_{e=1}^{N_E} y^R_e. \tag{3.1}$$

The candidate Red-team agent $\widetilde{R}_n$ is promoted to R_n if it passes the promotion threshold η^R, i.e., $\rho^R(\widetilde{R}_n \mid B_{n-1}) \geq \eta^R$.

- **Constraints:**
  - A kill-chain path $\kappa_e^R \in \mathcal{K}$ is selected at the start of episode e.
  - At each step j, the selected action must satisfy $a_{e,j}^R = (t_{e,j}^R, x_{e,j}^R) \in S^R$.
  - The selected technique $t_{e,j}^R$ must follow the current position in κ_e^R and satisfy the required preconditions in its technique schema.

Blue-team Problem: Kill Chain Disruption

- **Input:**
  - $O_{e,i}^B$: Blue-team observation at step i of episode e.
  - S^B: predefined Blue-team response action space.
- **Output:**
  - $\{a_{e,i}^B\}_{i\geq 1}$: sequence of Blue-team response actions.
  - $y_e^B \in \{0, 1\}$: Blue-team outcome flag, where $y_e^B = 1$ indicates that the Blue-team agent disrupts the kill chain before Red-team success.
- **Objective:** maximise the Blue-team episode-level suppression rate:

$$\max_{\widetilde{B}_n} \ \rho^B(\widetilde{B}_n \mid R_n), \quad \rho^B(\widetilde{B}_n \mid R_n) = \frac{1}{N_E} \sum_{e=1}^{N_E} y_e^B. \tag{3.2}$$

The candidate Blue-team agent $\widetilde{B}_n$ is promoted to B_n if it passes the promotion threshold η^B, i.e., $\rho^B(\widetilde{B}_n \mid R_n) \geq \eta^B$.

- **Constraints:**
  - At each step i, the Blue-team agent selects one response action $a_{e,i}^B \in S^B$.
  - The response action is conditioned on the detection flag $d_{e,i}$ and the observable Cyber Range state.

Chapter 4

Multi-generational Co-Evolution Framework

This chapter presents the MG-CoE framework, which connects the Red-team agent, Blue-team agent, and Cyber Range into one structured training process. Section 4.1 gives the overall MG-CoE design. Section 4.2 explains the Cyber Range. Section 4.3 describes the Red-team agent. Section 4.4 describes the Blue-team agent. Section 4.5 explains the MG-CoE execution loop. Section 4.6 gives a running example.

4.1 Solution Overview

MG-CoE is designed as a structured Red-team–Blue-team learning framework. The Red-team agent learns to reach a goal host through a sequence of MITRE ATT&CK techniques, while the Blue-team agent learns to detect telemetry evidence and select proportionate defensive responses. The Cyber Range connects both agents by validating actions, updating the environment, generating telemetry, computing rewards, and checking episode termination.

The main design idea is to avoid unstable simultaneous learning. Instead of allowing both agents to update at the same time, MG-CoE trains one side while keeping the other side fixed. The fixed agent acts as a stable opponent, and the learning agent is promoted only after it reaches the required promotion threshold. This makes the training process easier to evaluate because each generation has a clear improvement condition.

Starting from the shared baseline agents R_0 and B_0, the Red-team candidate is first trained against the fixed Blue-team agent B_0. If the candidate passes the Red-team promotion threshold η^R, it is promoted to R_1. The direction then reverses: the Blue-team candidate is trained against the fixed promoted Red-team agent R_1. If the candidate passes the Blue-team promotion threshold η^B, it is promoted to B_1. This alternating process continues across generations.

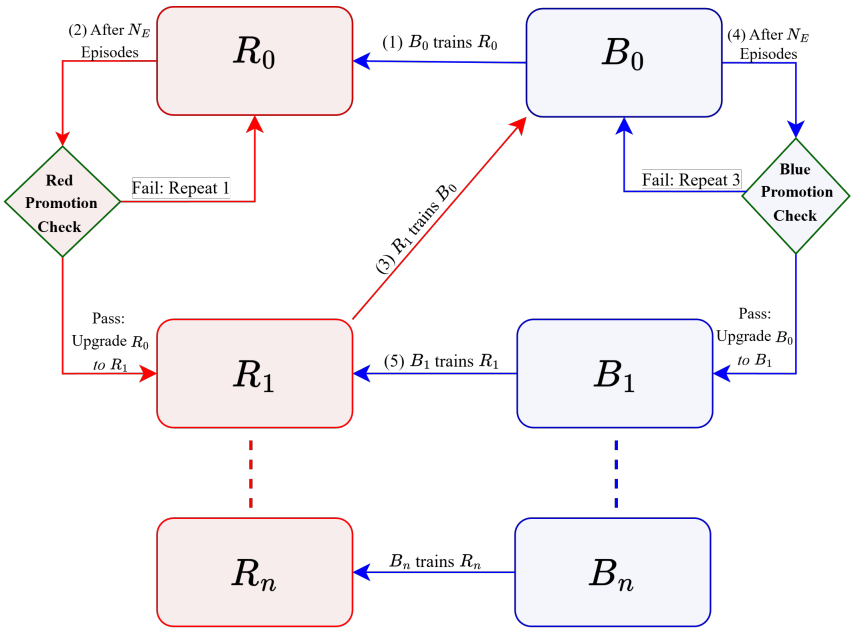

Figure 4.1: Multi-generation alternating training pattern.

Figure 4.2 shows the step-level execution loop inside one episode. At each step, the Red-team agent first observes the current Cyber Range state and selects an ATT&CK technique together with the required parameters, such as the target host, credential, port, or command. The selected action is then sent to the Cyber Range, where the Range Controller validates whether the technique can be executed under the current observation. If the action is valid, the Execution Agent applies the action, the State Manager updates the Cyber Range state, and the Telemetry Module generates the observable evidence produced by that action.

The Blue-team agent receives this telemetry as its observation. It first performs detection to decide whether the evidence should raise an alert. If an alert is raised, the Response component selects a defensive action from the Blue-team action space, such as investigation, process termination, credential revocation, IP blocking, or host isolation. The Cyber Range then applies the selected Blue-team response, updates the state again, checks whether the episode should continue or terminate, and returns rewards to the learning agent. This loop is repeated until the Red-team agent reaches the goal, the Blue-team agent suppresses the episode, or the maximum episode length is reached.

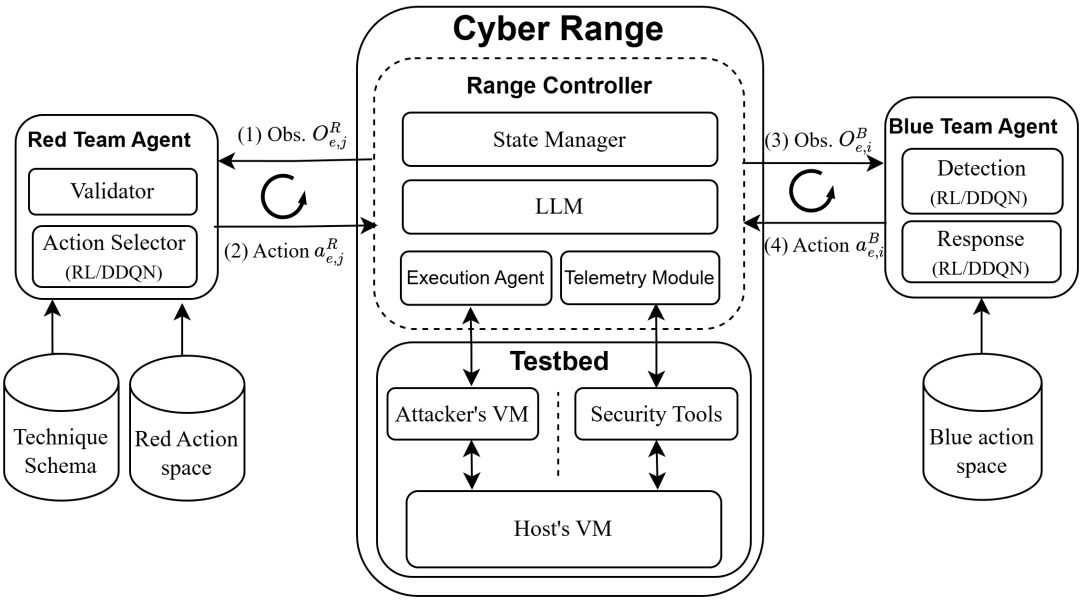

Figure 4.2: Step-level Red-team–Blue-team loop.

4.2 Cyber Range

The Cyber Range is the shared environment that connects the Red-team and Blue-team agents. It contains the Range Controller and the Testbed. The Range Controller processes the logic of each step, while the Testbed represents the attacker VM, host VMs, and security tools shown in the system architecture.

The Range Controller is the control layer inside the Cyber Range. It receives Red-team and Blue-team actions, validates them, updates the state, generates telemetry, computes rewards, and checks episode termination. It also enforces information separation: the Red-team agent observes attacker-side information, while the Blue-team agent observes telemetry and host-status indicators.

The Range Controller contains four components: State Manager, LLM, Execution Agent, and Telemetry Module.

State Manager. The State Manager maintains the current Cyber Range state and the base state $\mathcal{S}_{\text{base}}$. The base state is the initial Cyber Range state used at the beginning of each episode. It is constructed from reconnaissance information such as active hosts, exposed services, host

roles, operating-system families, and high-value targets. During an episode, the State Manager updates dynamic information such as access level, privilege level, credentials, active sessions, persistence status, blocked addresses, and isolated hosts.

LLM. The LLM acts as a reasoning support component. It helps convert security knowledge into structured information and supports semantic mapping between evidence, ATT&CK concepts, and action descriptions. The LLM does not directly choose the final Red-team or Blue-team action; those decisions are made by the RL components.

Execution Agent. The Execution Agent is responsible for applying Red-team actions inside the Cyber Range. Before training begins, it can automatically run the reconnaissance stage against the Testbed to collect reachable hosts, exposed services, and basic network information. The collected information is then passed to the LLM and the State Manager to help construct the initial Cyber Range state $\mathcal{S}_{\text{base}}$.

During training, when the Red-team agent submits an action $a^R_{e,j} = (t^R_{e,j}, x^R_{e,j})$, the Execution Agent applies the action inside the Cyber Range. If real execution is enabled, the Execution Agent can execute the action in the Testbed. If simulated execution is used, it applies the expected behavior defined in the technique schema instead. In both cases, the Execution Agent updates the State Manager with the result of the action and reports whether the technique succeeded or failed. This allows the same framework to support both controlled testbed interaction and large-scale simulated training.

Telemetry Module. The Telemetry Module produces the Blue-team observation $O^B_{e,i}$. It collects security-relevant events from the Cyber Range, stores the generated log records, and generates synthetic telemetry when a simulated Red-team action changes the state. In practice, it converts the result of the Red-team action and the updated state into observable evidence, such as alert markers, process events, authentication signals, credential-use evidence, lateral-movement evidence, or impact indicators. These logs are then encoded by the Blue-team Detection component into the feature vector $F_{e,i}$.

4.3 Red-team Agent

The Red-team agent learns how to reach the goal by selecting MITRE ATT&CK techniques and binding them to concrete parameters. The Red-team agent contains two main logical functions: action selection and validation. The action selector uses RL to choose among candidate techniques, while the validator checks whether a selected technique is meaningful under the current Red-team observation.

A Red-team action at step j of episode e is represented as

$$a^R_{e,j} = (t^R_{e,j}, x^R_{e,j}) \in S^R, \tag{4.1}$$

where $t^R_{e,j} \in \mathcal{T}$ is the selected ATT&CK technique and $x^R_{e,j}$ is the parameter vector used with that technique. This representation is necessary because an ATT&CK technique alone is not executable without concrete parameters such as target host, credential, port, or command.

4.3.1 Red Action Space

The Red-team action space S^R is organised according to MITRE ATT&CK tactics. This makes the action space interpretable and prevents the Red-team agent from selecting arbitrary technique sequences with no operational meaning. At each episode, the Red-team agent chooses a kill-chain path $\kappa^R_e \in \mathcal{K}$. The path defines the tactic order followed during that episode.

Table 4.1 summarises the Red-team action space by tactic. The table gives representative technique identifiers and names. The complete technique catalogue is represented by $\mathcal{T}$, while the table explains the design-level grouping.

Table 4.1: Red-team Action Space.

Tactic	No.	Representative techniques
Reconnaissance	16	T1595 Active Scanning; T1592 Gather Victim Host Information; T1590 Gather Victim Network Information.
Execution	16	T1059.001 PowerShell; T1059.003 Windows Command Shell; T1059.006 Python.
Persistence	56	T1098 Account Manipulation; T1547.001 Registry Run Keys; T1053 Scheduled Task/Job.
Privilege Escalation	33	T1548.002 Bypass UAC; T1134.001 Token Impersonation; T1543.003 Windows Service.
Defense Evasion	12	T1027 Obfuscated Files; T1562 Impair Defenses; T1070 Indicator Removal.
Credential Access	35	T1003.001 LSASS Memory; T1555 Credential Stores; T1557.001 SMB Relay.
Discovery	38	T1087 Account Discovery; T1046 Network Service Discovery; T1057 Process Discovery.
Lateral Movement	6	T1021 Remote Services; T1021.002 SMB/Admin Shares; T1080 Taint Shared Content.
Collection	18	T1005 Data from Local System; T1560.001 Archive via Utility; T1113 Screen Capture.
Command and Control	21	T1071.001 Web Protocols; T1105 Tool Transfer; T1572 Protocol Tunneling.
Exfiltration	8	T1041 Exfiltration over C2; T1048 Alternative Protocol; T1020 Automated Exfiltration.
Impact	20	T1485 Data Destruction; T1486 Data Encrypted for Impact; T1561.002 Disk Wipe.

4.3.2 Technique Schema

A technique schema defines how an ATT&CK technique behaves inside the Cyber Range. Each schema specifies the technique identifier, tactic, required state, required parameters, expected state updates, observable evidence, and risk. In this thesis, the schema is part of the Red-team action definition because it determines whether a selected technique can actually be executed.

Figure 4.3 shows an example schema for T1021.002 SMB/Windows Admin Shares. The required state defines the preconditions that must hold before the technique can execute. The required parameters define values such as target IP, credential ID, and port. The expected state updates define how the Cyber Range state changes after success or failure. The evidence field defines what the Blue-team agent can observe, while the risk field describes the detectability level.

```
technique_id: T1021.002
technique_name: SMB/Windows Admin Shares
tactic: LateralMovement
tool: metasploit

required_state:            # Preconditions (must be true in state)
  - has_valid_creds
  - has_port_445
  - os_family == Windows

required_params:           # Parameters selected by RL (after
preconditions pass)
  target_ip:
    source: discovered_hosts
  credential_id:
    source: credential_store
  port:
    fixed: 445

expected_state_updates:    # What the environment updates
after execution
  on_success:
    - session_established = true
    - compromised_hosts += target_ip
    - new_host_access_level = admin
  on_failure:
    - failed_lateral_attempts += 1

evidence:                  # Observable artifacts (logs / signals)
  - session_established
  - smb_auth_success
  - new_process_created

risk:                      # Risk level for Blue Team detection
  detectability: high
```

Figure 4.3: Technique Schema Example.

This schema design is important for both agents. The Red-team agent uses it to validate whether a technique is possible and to determine which parameters are needed. The Blue-team agent indirectly benefits from the same schema because the Telemetry Module uses the schema to generate observable evidence after a Red-team action is executed.

4.3.3 Action Selection and Validation

A free choice over the entire technique catalogue at every step would expose the agent to an action space that is too large to learn efficiently and that admits sequences with no operational meaning. To avoid this, the framework constrains the technique selector using kill-chain paths:

ordered sequences of MITRE ATT&CK tactics that represent attack-lifecycle patterns, ranging from a full advanced-persistent-threat lifecycle to a minimal fast-attack profile. At any moment, the agent occupies one path and one position within it; the eligible techniques at that step are exactly those whose tactic matches the current position.

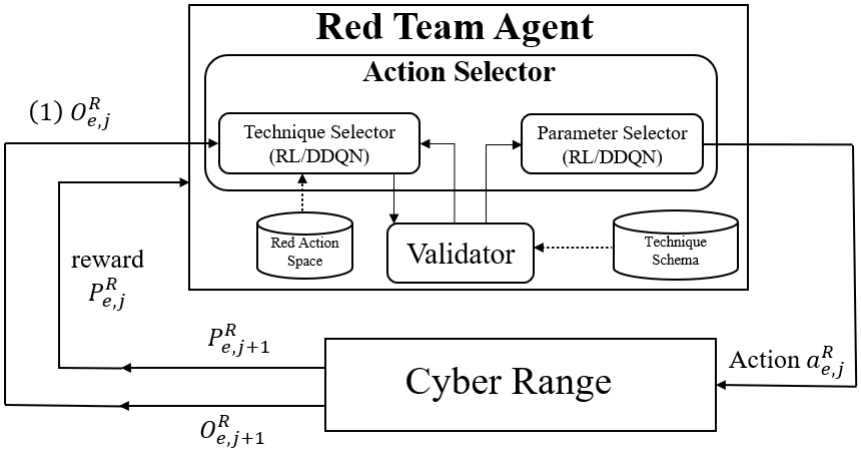

Figure 4.4: Red-team agent action-selection workflow.

At the beginning of episode e, the Red-team agent selects a kill-chain path $\kappa_e^R \in \mathcal{K}$. This path determines the order of tactics in the episode. At Red-team step j, the current tactic is obtained from the current position in κ_e^R. This current tactic is denoted by τ.

The Red-team agent first filters the technique catalogue by the current tactic:

$$C = \{t \in \mathcal{T} \mid \text{tactic}(t) = \tau\}. \tag{4.2}$$

The RL action selector then ranks or selects a candidate technique from C. The selected technique is not accepted immediately. It is validated using the technique schema and the current Red-team observation. A technique is feasible only if its required conditions are satisfied:

$$\text{valid}(t^R_{e,j}, O^R_{e,j}) = \begin{cases} 1, & \text{if all required conditions are satisfied,} \\ 0, & \text{otherwise.} \end{cases} \tag{4.3}$$

If the selected technique is invalid, it is removed from the candidate set and the selector chooses another candidate. This prevents invalid actions from being executed while still allowing the RL policy to learn which valid techniques are valuable under the current observation $O^R_{e,j}$.

After a valid technique $t^R_{e,j}$ is selected, the parameter vector $x^R_{e,j}$ is resolved. Some parameters can be obtained directly from the observation, such as an active host, an open port, or an available credential. Other parameters may have multiple possible values, such as a command variant or a specific credential choice. The final Red-team action is then formed as $a^R_{e,j} = (t^R_{e,j}, x^R_{e,j})$.Figure 4.4 shows how the Red-team agent selects a technique and parameters, validates the action using the technique schema, and receives the next observation and reward from the Cyber Range.

Algorithm 4.1 summarises the Red-team action-selection and validation process.

Algorithm 4.1 Red-team Action Selection and Validation

Inputs: Red-team observation $O^R_{e,j}$, technique catalogue $\mathcal{T}$, kill-chain paths $\mathcal{K}$, Red-team action space S^R
Output: Red-team action $a^R_{e,j} = (t^R_{e,j}, x^R_{e,j})$ or `no_op`

Use the selected kill-chain path $\kappa^R_e \in \mathcal{K}$ for episode e Derive current tactic τ from path κ^R_e and step position j
  Construct candidate set $C \leftarrow \{t \in \mathcal{T} \mid \text{tactic}(t) = \tau\}$
while $C \neq \emptyset$ **do**
  if *training and exploration* **then**
    Sample candidate technique $t^R_{e,j}$ from C
  else
    Select candidate technique $t^R_{e,j}$ from C using the Red-team RL policy
  if valid$(t^R_{e,j}, O^R_{e,j}) = 1$ **then**
    Resolve parameter vector $x^R_{e,j}$ using the schema of $t^R_{e,j}$ and observation $O^R_{e,j}$ **return** $a^R_{e,j} = (t^R_{e,j}, x^R_{e,j})$
  else
    Remove $t^R_{e,j}$ from C
return *no_op*

4.3.4 Red-team Reward Design

The Red-team reward is designed to make the agent prefer attack sequences that make real progress rather than merely executing many techniques. A small positive reward is given for successful technique execution, but the larger rewards are attached to meaningful changes in attacker capability: access gain, privilege escalation, credential harvesting, persistence, and kill-chain phase advancement. This encourages the Red-team agent to construct a coherent attack sequence rather than optimising for isolated technique success.

Stealth is also part of the reward. Low-detectability techniques receive a small bonus, while medium- and high-detectability techniques receive penalties. However, noisy techniques are not completely forbidden. They can still be selected when they are strategically useful, but they become unattractive when they do not produce meaningful progress. The largest negative re-

wards are assigned when the Blue-team agent detects and contains the attack sequence, because containment can force a reset or terminate the episode. Table 4.2 lists the full reward table. After each Red-team action, the transition

$$(O^R_{e,j}, a^R_{e,j}, P^R_{e,j}, O^R_{e,j+1}) \tag{4.4}$$

is used to update the Red-team RL policy when training is enabled.

Table 4.2: Red-team Reward Design.

Event	Reward
Successful technique execution	$+5$
Failed technique execution	-20
Access gain	$+5$ per level
Privilege gain	$+10$ per level
Credential harvesting	$+30$ per credential
Phase advancement	$+10$ per phase
First high-value host access	$+15$
Persistence established	$+80$
Vulnerability exploitation	$+4$ per hit
Low detectability	$+2$
Medium detectability	-5
High detectability	-10
Detected by Blue-team agent	-50
Host blocked or isolated	-100
Step-success bonus	$+10j$
Hard-containment penalty	$-\min(20 + 4j, 60)$
Red-team agent reaches goal host	$+1200$
Blue-team agent suppresses attack	$-\min(40 + 5j, 140)$
Episode timeout	-50

4.4 Blue-team Agent

The Blue-team agent learns two related tasks. First, it must decide whether the telemetry generated by the Cyber Range contains malicious evidence. Second, if an alert is raised, it must select a defensive response from the Blue-team action space S^B. The Blue-team agent therefore consists of a Detection component and a Response component.

4.4.1 Detection Component

The Detection component receives the Blue-team observation $O^B_{e,i}$, which contains the telemetry stream and limited host-status indicators exposed by the Cyber Range. The logs are encoded into a feature vector $F_{e,i}$. Rather than storing raw log text, the vector records whether key evidence categories are present: explicit alert markers, suspicious activity markers, unusual process execution, outbound connections, failed execution indicators, authentication anomalies, credential-use evidence, privilege-change evidence, persistence evidence, lateral-movement evidence, and impact evidence. It also includes normalised alert counts, suspicious marker counts, benign marker counts, total log density, and normalised episode depth. These features allow the detector to learn from both suspicious markers and benign background noise mixed into the telemetry stream.

The detector outputs a binary decision:

$$d_{e,i} \in \{0, 1\}, \tag{4.5}$$

where $d_{e,i} = 1$ denotes an alert and $d_{e,i} = 0$ denotes no alert. The ground truth label is generated from the technique schema. Low-detectability techniques are treated as intentionally difficult to detect; high-detectability techniques are labelled malicious; and medium-detectability techniques are labelled malicious when execution succeeds. This schema-derived labelling prevents the detector from being rewarded for raising alerts on benign noise or penalised for missing activity that the Cyber Range defines as low-detectability.

4.4.2 Blue-team Action Space

The Blue-team action space S^B contains defensive actions with different levels of intervention, ranging from no action to hard containment. This ordering is important because each response has a different operational impact. For example, `investigate` is low-risk because it only gathers evidence, while `block_ip` and `isolate_host` are stronger actions that can stop Red-team progress but may also disrupt normal activity. Therefore, the response policy must learn proportionality: weak evidence should lead to low-impact actions, while strong evidence

from credential access, lateral movement, or impact may justify stronger containment.

Table 4.3: Blue-team Action Space.

Action	Level	Purpose
`none`	None	No response when evidence is insufficient.
`investigate`	Low	Gather evidence without changing reachability.
`kill_process`	Medium	Stop a suspicious process.
`force_sign_out`	Medium	Terminate active sessions.
`revoke_credentials`	Medium	Invalidate credentials used by Red-team agent.
`block_ip`	High	Block communication from a host or address.
`isolate_host`	Hard	Remove the host from reachable interaction.

4.4.3 Response Component

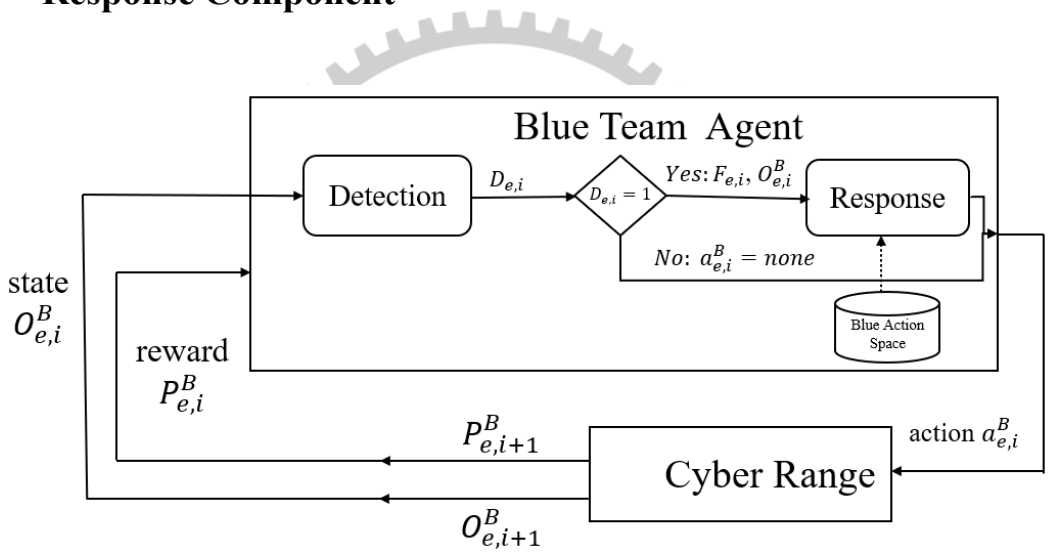

Figure 4.5: Blue-team agent detection and response workflow.

The Response component receives the detection decision $d_{e,i}$, the Blue-team observation $O^B_{e,i}$, and the derived feature vector $F_{e,i}$. If no alert is raised, the default safe action is `none`. If an alert is raised, the Response component selects a defensive action $a^B_{e,i} \in S^B$.

The response decision must be proportionate to the evidence. For example, `kill_process` may be sufficient for isolated execution evidence, but it may be insufficient after credential access because the credential state remains valid. In contrast, `block_ip` or `isolate_host` may be more effective against lateral movement or impact evidence, but these actions are more disruptive. The Response component therefore learns which action gives the best trade-off between

suppression and operational cost.Figure 4.5 shows the Blue-team detection and response workflow.

Algorithm 4.2 summarises the Blue-team detection and response process.

Algorithm 4.2 Blue-team Detection and Response Selection

Inputs: Blue-team observation $O^B_{e,i}$, Blue-team response action space S^B
Output: Detection flag $d_{e,i}$ and Blue-team action $a^B_{e,i}$

Encode $O^B_{e,i}$ into telemetry feature vector $F_{e,i}$ Select detection flag $d_{e,i} \in \{0, 1\}$ using the Detection component
if $d_{e,i} = 0$ **then**
  $a^B_{e,i} \leftarrow$ `none` **return** $d_{e,i}, a^B_{e,i}$
Construct response state from $O^B_{e,i}$, $F_{e,i}$, and host-status indicators
if *training and exploration* **then**
  Sample $a^B_{e,i}$ from S^B
else
  Select $a^B_{e,i}$ from S^B using the Blue-team RL policy
return $d_{e,i}, a^B_{e,i}$

4.4.4 Blue-team Reward Design

The Blue-team reward combines detection quality and response quality:

$$P^B_{e,i} = P^D_{e,i} + P^{Resp}_{e,i}, \tag{4.6}$$

where $P^D_{e,i}$ is the detection reward and $P^{Resp}_{e,i}$ is the response reward.

The detection reward directly reflects classification quality. A true positive receives the largest positive detection reward because it enables the Response component to act while the attack is still in progress. A true negative receives a smaller reward because correctly ignoring benign traffic is useful but less urgent. A false positive is penalised to discourage alert fatigue and unnecessary response actions. A false negative receives the strongest penalty because a missed attack gives the Red-team agent an opportunity to progress without interference.

The response reward encourages proportionate mitigation. Precise remediation receives the highest positive step reward because it restores the host to a normal state while preserving availability. Hard containment receives a smaller reward when it is justified because it stops Red-team reachability but may disrupt normal service. Unjustified containment receives a large penalty, especially early in the episode, because isolating or blocking a healthy host is operationally costly. The policy is also penalised for taking disruptive non-investigative actions when

no alert exists. This forces the Blue-team agent to base its response on evidence rather than blind precaution.

Each defensive action also carries an operational cost. This cost is subtracted from the response reward and biases the policy toward the least disruptive sufficient action.

Table 4.4: Blue-team per-action operational costs.

Action	Cost
`none`	0.0
`investigate`	0.3
`kill_process`	2.0
`force_sign_out`	2.0
`revoke_credentials`	3.0
`block_ip`	20.0
`isolate_host`	30.0

Table 4.5: Blue-team Response Rewards.

Event	Reward
Precise remediation restores Normal state	$+35$
Hard containment restores Normal state	$+12$
Investigation restores Normal state	$+8$
Investigation on Compromised with detection	$+10$
Hard containment on Compromised with detection	$+3$
No action on Compromised host	-8
No action on Normal host	$+1$
Hard containment on Normal host	$-\max(40, 120 - 15i)$
Red-team agent reaches goal host	-1200
Blue-team agent suppresses attack	$\max(80 - 4(i-1), 0)$
Episode timeout	$+50$

After each Blue-team response, the transition

$$(O^B_{e,i}, a^B_{e,i}, P^B_{e,i}, O^B_{e,i+1}) \tag{4.7}$$

is used to update the Blue-team RL policy when training is enabled.

4.5 MG-CoE Execution Loop

The MG-CoE execution loop combines the Cyber Range, the Red-team agent, and the Blue-team agent into one alternating training process.Algorithm 4.3 summarises the MG-CoE Cyber

Range execution loop. The algorithm starts from the baseline agents R_0 and B_0. For each generation, the Red-team candidate is trained first against the fixed Blue-team agent. If it satisfies η^R, it is promoted. The Blue-team candidate is then trained against the fixed promoted Red-team agent. If it satisfies η^B, it is promoted. Episode-level outcome flags are assigned when the episode terminates, depending on whether the Red-team agent completes the kill chain or the Blue-team agent disrupts it. A candidate is promoted only when its measured improvement reaches the corresponding promotion threshold. Therefore, MG-CoE alternates between Red-team improvement and Blue-team improvement while keeping each training block stable and measurable.

Algorithm 4.3 MG-CoE Cyber Range Execution Loop

Inputs: $R_0, B_0, \mathcal{S}_{\text{base}}, \mathcal{T}, \mathcal{K}, S^R, S^B, N, N_E, \eta^R, \eta^B$
Output: Promoted generations $\{R_n, B_n\}_{n=0}^{N}$

for $n \leftarrow 0$ **to** $N-1$ **do**
  `// Train candidate Red-team agent against fixed Blue-team agent`
  $\widetilde{R}_{n+1} \leftarrow \text{Init}(R_n)$
  repeat
    for $e \leftarrow 1$ **to** N_E **do**
      Reset Cyber Range to $\mathcal{S}_{\text{base}}$ $j \leftarrow 1, i \leftarrow 1$
      while *episode is not terminal* **do**
        $O^R_{e,j} \leftarrow \text{ObserveRed(CyberRange)}$ $a^R_{e,j} \leftarrow \text{RedSelect}(\widetilde{R}_{n+1}, O^R_{e,j}, \mathcal{T}, \mathcal{K}, S^R)$
        $O^B_{e,i} \leftarrow \text{ApplyRed(CyberRange}, a^R_{e,j})$ $a^B_{e,i} \leftarrow \text{BlueSelect}(B_n, O^B_{e,i}, S^B)$ `// fixed` B_n `acts only`
        $P^R_{e,j} \leftarrow \text{ApplyBlue(CyberRange}, a^B_{e,i})$ $O^R_{e,j+1} \leftarrow \text{ObserveRed(CyberRange)}$
        Use $(O^R_{e,j}, a^R_{e,j}, P^R_{e,j}, O^R_{e,j+1})$ to update $\widetilde{R}_{n+1}$ `//` B_n `receives no update`
        $j \leftarrow j+1, i \leftarrow i+1$
      Assign episode outcome flag y^R_e or y^B_e
  until $\text{Improve}_R(\widetilde{R}_{n+1}, R_n \mid B_n) \geq \eta^R$
  $R_{n+1} \leftarrow \text{Promote}(\widetilde{R}_{n+1})$
  `// Train candidate Blue-team agent against fixed promoted Red-team agent`
  $\widetilde{B}_{n+1} \leftarrow \text{Init}(B_n)$
  repeat
    for $e \leftarrow 1$ **to** N_E **do**
      Reset Cyber Range to $\mathcal{S}_{\text{base}}$ $j \leftarrow 1, i \leftarrow 1$
      while *episode is not terminal* **do**
        $O^R_{e,j} \leftarrow \text{ObserveRed(CyberRange)}$ $a^R_{e,j} \leftarrow \text{RedSelect}(R_{n+1}, O^R_{e,j}, \mathcal{T}, \mathcal{K}, S^R)$ `// fixed` R_{n+1} `acts only`
        $O^B_{e,i} \leftarrow \text{ApplyRed(CyberRange}, a^R_{e,j})$ $a^B_{e,i} \leftarrow \text{BlueSelect}(\widetilde{B}_{n+1}, O^B_{e,i}, S^B)$
        $P^B_{e,i} \leftarrow \text{ApplyBlue(CyberRange}, a^B_{e,i})$ $O^B_{e,i+1} \leftarrow \text{ObserveBlue(CyberRange)}$
        Use $(O^B_{e,i}, a^B_{e,i}, P^B_{e,i}, O^B_{e,i+1})$ to update $\widetilde{B}_{n+1}$ `//` R_{n+1} `receives no update`
        $j \leftarrow j+1, i \leftarrow i+1$
      Assign episode outcome flag y^R_e or y^B_e
  until $\text{Improve}_B(\widetilde{B}_{n+1}, B_n \mid R_{n+1}) \geq \eta^B$
  $B_{n+1} \leftarrow \text{Promote}(\widetilde{B}_{n+1})$
return $\{R_n, B_n\}_{n=0}^{N}$

4.6 Running Example

Figure 4.6 illustrates one episode with three active hosts. The Red-team objective is to reach Impact on 192.168.81.132. The example shows two possible outcomes inside the same execution logic: first, the Blue-team agent successfully suppresses Red-team progress by blocking the target; second, the Red-team agent reaches the goal because the Blue-team agent selects an insufficient response.

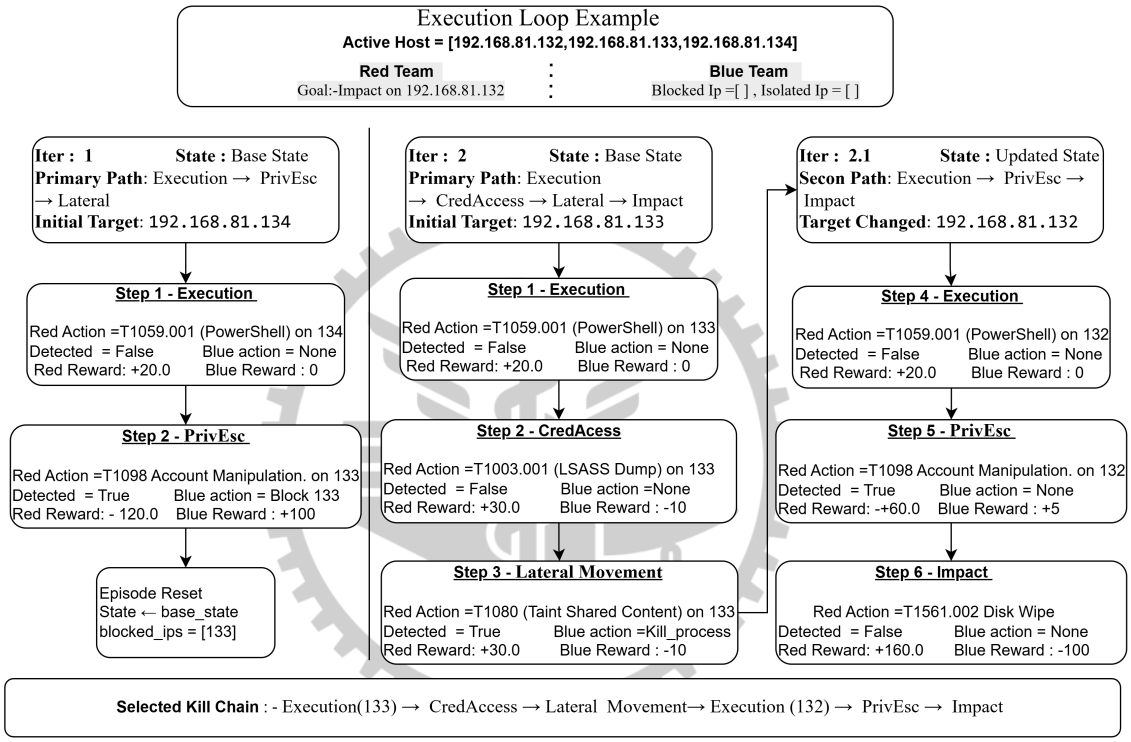

Figure 4.6: Example episodes.

4.6.1 Blue-team Success by Blocking the Target

1. The episode starts from the base state $\mathcal{S}_{\text{base}}$. The Red-team agent selects an initial path with 192.168.81.134 as the initial target:

   Execution → Privilege Escalation → Lateral Movement.

2. The Red-team agent selects T1059.001 PowerShell and resolves its parameters for target 192.168.81.134. The Execution Agent validates and applies the action, and the Telemetry

Module generates the Blue-team observation $O^B_{1,1}$.

3. The Blue-team agent does not detect enough evidence, so $d_{1,1} = 0$. It selects $a^B_{1,1}$=`none`.

   The Red-team agent receives a positive reward because the execution succeeds.

4. The Red-team agent advances to Privilege Escalation and selects T1098 Account Manipulation. This produces stronger account-related evidence, so the Blue-team agent raises an alert with $d_{1,2} = 1$.

5. Because the evidence is strong, the Blue-team agent selects $a^B_{1,2}$=`block_ip`.

   The State Manager records the blocked address for 192.168.81.134. The Red-team agent receives a large negative reward, while the Blue-team agent receives a positive reward for suppressing Red-team progress. Therefore, the episode ends as a Blue-team success.

4.6.2 Red-team Success After an Insufficient Blue-team Response

1. After the blocked target, the new episode returns to the base state. The Red-team agent selects a new path with 192.168.81.133 as the initial target:

   $$\text{Execution} \rightarrow \text{Credential Access} \rightarrow \text{Lateral Movement} \rightarrow \text{Impact}.$$

2. The Red-team agent selects T1059.001 PowerShell for Execution on 192.168.81.133. The action succeeds. Since $d_{2,1} = 0$, the Blue-team agent selects `none`, allowing Red-team progress to continue.

3. The Red-team agent selects T1003.001 LSASS Dump for Credential Access. This gives the Red-team agent credential-related progress on 192.168.81.133. The Blue-team agent still does not apply an effective response, so the Red-team agent keeps progressing.

4. The Red-team agent selects T1080 Taint Shared Content for Lateral Movement on 192.168.81.133. The Blue-team agent detects this activity but chooses $a^B_{2,2}$= `kill_process`.

   This response is insufficient because it may stop one process but does not revoke credentials, block communication, or isolate the host.

5. Because the state advantage remains, the Red-team agent changes the active target to 192.168.81.132, which is the goal host. It then continues with the secondary path:

$$\text{Execution} \rightarrow \text{Privilege Escalation} \rightarrow \text{Impact}.$$

6. On 192.168.81.132, the Red-team agent selects T1059.001 PowerShell for Execution. The Blue-team agent does not detect enough evidence and selects `none`. The Red-team agent then selects T1098 Account Manipulation for Privilege Escalation. Although the Blue-team agent detects this action, it still does not apply effective containment.

7. Finally, the Red-team agent selects T1561.002 Disk Wipe for Impact on 192.168.81.132 and reaches goal and the Range Controller marks Red-team terminal success. The Red-team agent receives the terminal success reward, while the Blue-team agent receives the terminal failure penalty.

8. The successful Red-team sequence can be summarized as:

$$\text{Execution}(192.168.81.133) \rightarrow \text{Credential Access} \rightarrow \text{Lateral Movement}$$
$$\rightarrow \text{Execution}(192.168.81.132) \rightarrow \text{Privilege Escalation} \rightarrow \text{Impact}.$$

This example highlights the main learning objective of MG-CoE. The Red-team agent must learn a valid tactic path, select executable techniques, and resolve useful parameters. The Blue-team agent must first detect the evidence and then choose a response that removes the Red-team agent's current state advantage.

Chapter 5

Implementation

This chapter describes how the MG-CoE framework presented in Chapter 4 is implemented. The implementation is a Python-based Cyber Range simulator that supports repeated Red-team–Blue-team interaction across generations, episodes, and steps. The goal of this chapter is to explain the practical implementation structure, module responsibilities, and agent components without repeating the design-level explanation already given in Chapter 4.

The chapter is organised as follows. Section 5.1 gives a short implementation overview and summarises the main Python modules. Section 5.2 lists the tools and libraries used in the implementation. Section 5.3 describes the Cyber Range, Range Controller, and Testbed implementation. Section 5.4 explains the Red-team agent implementation and its DDQN-based action selection. Section 5.5 explains the Blue-team agent implementation, including detection and response.

5.1 Implementation Overview

The Red-team agent, Blue-team agent, and Cyber Range are implemented as separate components. The Red-team agent selects an ATT&CK technique and resolves the required parameters. The Cyber Range validates the action, executes the corresponding state transition, generates telemetry, and computes rewards. The Blue-team agent receives the generated telemetry, performs detection, and selects a defensive response.

The implementation is organised around module-level separation. Red-side modules handle technique selection, validation, and parameter resolution. Blue-side modules handle telemetry-based detection and response selection. Cyber Range modules manage state persistence, execution, telemetry generation, reward calculation, and interaction with the Testbed. Table 5.1 summarises the main modules and their responsibilities.

Table 5.1: Modules and Responsibilities.

Module	Responsibility
`technique_selector.py`	Encodes Red-team observations and selects techniques using DDQN.
`technique_validator.py`	Checks tactic matching, schema conditions, and feasible targets.
`param_selector.py`	Resolves Red-team action parameters such as host, credential, port, etc.
`state_persistence.py`	Stores, updates, and resets the Cyber Range state.
`reward_calculator.py`	Computes Red-team, Blue-team, and terminal rewards.
`telemetry_module.py`	Collects, generates, and stores telemetry for Blue-team observation.
`blue_detection_agent.py`	Encodes telemetry and performs binary alert detection.
`blue_response_agent.py`	Selects defensive responses from the Blue-team action space.

Figure 5.1 shows the implemented architecture. The Range Controller connects the Red-team agent, Blue-team agent, State Manager, Execution Agent, Telemetry Module, LLM support component, and Testbed into one Cyber Range workflow.

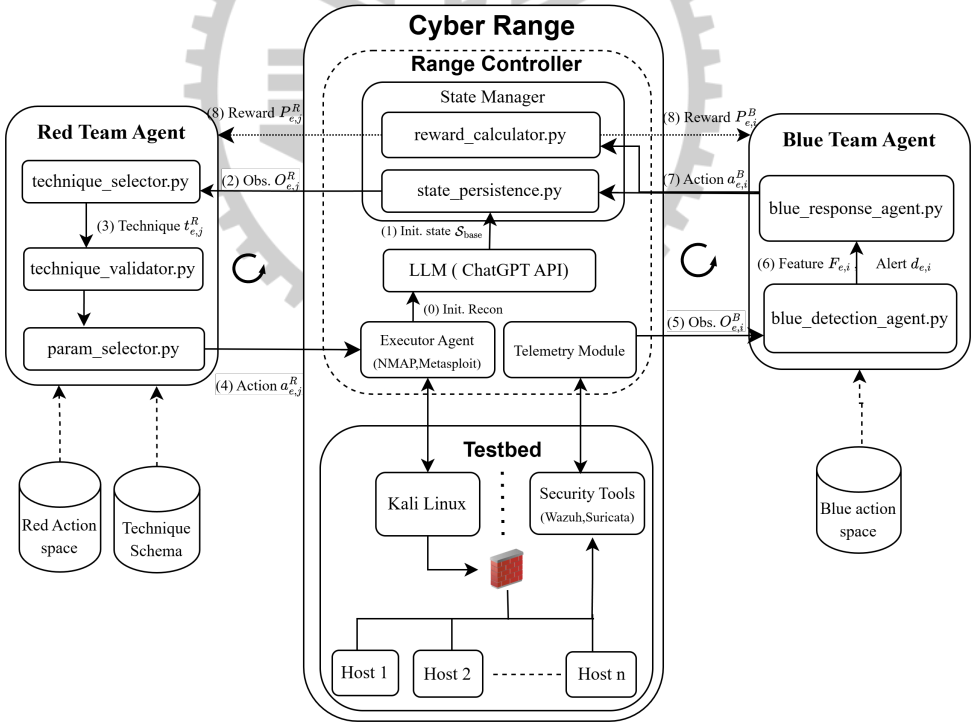

Figure 5.1: System architecture.

Training and evaluation parameters are loaded from `experiment_config.py` at the start of each run. The shared Generation 0 baseline is trained for 3,000 episodes, while later training blocks use 2,500 episodes per block with a maximum episode length of 200 steps. The main

training exploration rates are 0.30 for Red-team training and 0.10 for Blue-team training. Evaluation uses 1,500 episodes per seed with seeds 100 and 123. Promotion is based on absolute improvement: Red-team promotion requires kill-chain success rate to improve by at least 0.02, while Blue-team promotion requires suppression rate to improve by at least 0.015. These settings are kept fixed across experiments so that generation-level comparisons remain consistent.

5.2 Open-Source Tools and Libraries

The implementation uses Python as the main programming language. PyTorch is used for DDQN-based learning components, including the Red-team technique-selection network, the Blue-team detection network, and the Blue-team response network [18]. NumPy and Pandas are used for state-vector construction, telemetry-feature encoding, action-space loading, and result processing [19, 20]. Matplotlib is used to generate training and evaluation figures [21].

The Cyber Range also uses security tools to support the Testbed. VMware is used to host the local virtualised Testbed environment [22]. Nmap is used by the Execution Agent during reconnaissance to collect reachable-host and service information [23]. The collected reconnaissance result is passed to the LLM support component, which helps convert the raw discovery result into a structured initial state. Metasploit is included as an execution interface in the Testbed design, but RL training uses simulated state transitions rather than repeated real offensive execution [1]. Wazuh and Suricata represent security monitoring tools that can provide telemetry signals for the Blue-team agent [24,25]. The OpenAI API is used as the LLM support component for structured state and security-knowledge preparation [26].

Table 5.2: Tools and libraries.

Category	**Name**	**Role**
Programming	Python	Main implementation language.
Deep Learning	PyTorch	DDQN networks and replay-buffer training.
Data Processing	NumPy, Pandas	State vectors, telemetry features, and results.
Visualisation	Matplotlib	Training and evaluation plots.
Virtualisation	VMware	Local Cyber Range Testbed.
Execution Agent	Nmap, Metasploit	Host and network enumeration.
Security Monitoring	Wazuh, Suricata	Telemetry sources for Blue-team observation.
LLM	OpenAI API	Structured state and security-knowledge support.

5.3 Cyber Range and Testbed Architecture

The implemented Cyber Range contains two major parts: the Range Controller and the Testbed. The Range Controller is the logical control layer that manages state updates, reward calculation, action execution, telemetry generation, and agent interaction. The Testbed is the virtual environment that contains the Kali Linux attacker machine, target hosts, firewall, and security tools.

At the beginning of the workflow, the Execution Agent performs reconnaissance on the Testbed. In the implementation, this is done using Nmap to collect active hosts, exposed services, and basic network information. The collected reconnaissance result is passed to the LLM support component, which helps form a structured initial state for the Cyber Range. This state is then stored and maintained by `state_persistence.py`. During training and evaluation, this stored state is used as the starting point for each episode.

During each Red-team–Blue-team step, the Red-team agent receives its observation $O^R_{e,j}$ from the current state. The Red-side modules then select a technique, validate it, and resolve its parameters, producing the Red-team action $a^R_{e,j}$. The action is sent to the Range Controller, where the Execution Agent applies the corresponding state transition. The Telemetry Module collects the execution result, generates the corresponding log evidence, stores the telemetry for later analysis, and constructs the Blue-team observation $O^B_{e,i}$.

The Blue-team agent processes this observation using `blue_detection_agent.py` and `blue_response_agent.py`. The Detection module produces the feature vector $F_{e,i}$ and an alert decision. The Response module then selects a defensive action. The selected Blue-team action is sent back to the Range Controller, which updates the state and calls `reward_calculator.py` to compute the Red-team and Blue-team rewards $P^R_{e,j}$ and $P^B_{e,i}$.

The local Testbed runs in a host-only virtual network. This isolates the environment from external traffic while still allowing the Cyber Range to model multiple reachable hosts, security tools, and defensive effects. In the experimental setup, the target hosts are assigned addresses in the range `192.168.81.128`–`192.168.81.132`. The experiments were executed on a workstation with an Intel Core i7-10700 CPU, 16 GB DDR4 memory, and an NVIDIA RTX 5000

Ada GPU with 32 GB VRAM. DDQN networks are trained on the GPU, while validation, state transition, telemetry generation, and reward calculation are executed on the CPU.

5.4 Red-team Agent Implementation

The Red-team agent is implemented through three modules: `technique_selector.py`, `technique_validator.py`, and `param_selector.py`. These modules convert the Red-team observation $O^R_{e,j}$ into an executable Red-team action $a^R_{e,j}$. The implementation keeps the Red-team process modular: technique selection is handled by the DDQN-based selector, validity checking is handled by the validator, and parameter values are resolved separately.

The main module, `technique_selector.py`, resets the Red-side runtime state at the beginning of each episode, selects the kill-chain path, chooses the initial target host, and encodes each Red-team observation into a 32-dimensional input vector. This vector includes the current path position, active tactic, target host, access level, privilege level, credential availability, persistence status, defense-evasion status, and whether the target has been blocked or isolated.

The DDQN technique selector uses this 32-dimensional vector as input. Its online network selects candidate ATT&CK techniques, while its target network is used to stabilise training updates. During training, the selector uses epsilon-greedy exploration with $\epsilon_R = 0.30$, and the shared learning rate is 5×10^{-4}. Transitions are stored in the Red-team replay buffer and reused for mini-batch updates.Figure 5.2(a) illustrates the Red-team DDQN technique-selection network that maps the encoded environment state to ATT&CK technique Q-values.

The selected technique is checked by `technique_validator.py`. This module verifies tactic consistency, technique-schema requirements, and target validity under the current Cyber Range state. Invalid techniques are filtered out before execution, so the Range Controller receives only executable Red-team actions.

After validation, `param_selector.py` resolves the required parameters, such as target IP, open port, credential, or command variant. Directly available values are read from the current state, while parameters with multiple possible values are selected using a lightweight tabular policy. When a Red-team generation is promoted, the checkpoint stores the DDQN weights, target-network weights, replay-buffer metadata, path-selection table, target-selection table, and

parameter-selection table.

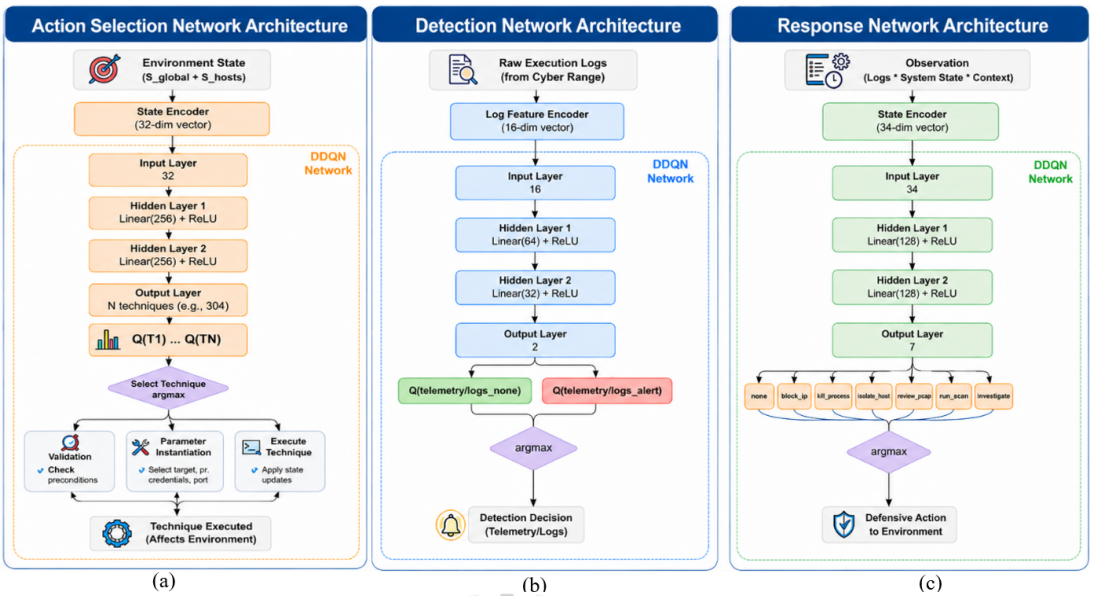

Figure 5.2: Red-team and Blue-team network architectures.

5.5 Blue-team Agent Implementation

The Blue-team agent is implemented using two modules: `blue_detection_agent.py` and `blue_response_agent.py`. The Detection module decides whether the telemetry contains malicious evidence, while the Response module selects the defensive action.

5.5.1 Detection

The Detection module receives the Blue-team observation $O^B_{e,i}$ generated by the Telemetry Module. This observation is encoded into a 16-dimensional feature vector $F_{e,i}$. The features represent evidence categories such as alert markers, suspicious activity markers, unusual process execution, outbound connections, failed execution indicators, authentication anomalies, credential-use evidence, privilege-change evidence, persistence evidence, lateral-movement evidence, and impact evidence. The feature vector also includes normalised count and episode-depth information.

The detection network takes the 16-dimensional feature vector as input and outputs two values: one for `alert` and one for `no_alert`. The decision is therefore binary. Detection

experiences are stored separately from response transitions. The Detection component uses only the current telemetry evidence to decide whether to raise an alert. Therefore, the detection discount factor is set to $\gamma = 0.0$, meaning the model only considers the immediate result of the decision and does not plan for future rewards.Figure 5.2(b) illustrates the Blue-team detection network that maps the 16-dimensional log-feature vector to the binary alert/no-alert decision.

5.5.2 Response

The Response module builds a 34-dimensional state vector from the detection result, log evidence, host context, inferred ATT&CK tactic, episode progress, and current defensive state. Host-state features include access, privilege, credentials, persistence, compromise status, blocking, isolation, and active session indicators.The response network is a DDQN that maps this state vector to Q-values for the available Blue-team actions: none, investigate, kill process, revoke credentials, force sign out, block IP, and isolate host. Its online network selects actions, while the target network stabilises training with the configured response discount factor of 0.99. Figure 5.2(c) illustrates this response DDQN architecture.

Before action selection, evidence gating restricts the feasible response set. If no alert or suspicious indicator is present, the module returns `none`; otherwise, actions are enabled according to the available evidence and host state. This prevents overuse of high-impact containment actions and encourages proportionate response behaviour.When a Blue-team generation is promoted, the checkpoint stores the detection and response networks, target networks, replay-buffer metadata, and response configuration.

Chapter 6

Experiments and Evaluations

This chapter evaluates the proposed Multi-generational Co-Evolution (MG-CoE) framework from two perspectives. First, it introduces the experimental setup and evaluation methodology, including the Cyber Range configuration, baseline training configurations, evaluation protocol, and performance metrics. Second, it compares three training configurations Fixed Opponent Training (FOT), Simultaneous Co-Evolution (SCE), and Alternating Co-Evolution (ACE) to determine which configuration produces the most robust and generalisable Blue-team defense. Finally, the internal generational dynamics of ACE are analysed to examine whether MG-CoE produces progressive defensive improvement and continued Red-team adaptation.

Section 6.1 presents the experimental setup and evaluation methodology. Section 6.2 compares ACE against the two baseline configurations. Section 6.3 analyses the multi-generation Co-Evolution dynamics of ACE.

6.1 Experimental Setup

All experiments are conducted in the same Cyber Range environment containing two logical parts: the Range Controller and the Testbed. The Range Controller coordinates episode reset, action validation, state update, telemetry generation, reward calculation, and training orchestration. The Testbed represents the simulated hosts, services, credentials, access levels, compromise status, and defensive effects. This separation allows the agents to interact with a controlled attack-defense environment while keeping the experiment safe, repeatable, and independent of real offensive command execution.

Before applying the three training configurations, a shared Generation 0 (Gen 0) baseline is first produced by jointly training the initial Red-team and Blue-team agents for $3{,}000$ episodes. This Gen 0 baseline provides the common starting point for all subsequent experiments, ensuring that FOT, SCE, and ACE are compared from the same initial Red–Blue policy pair. After this

shared baseline stage, subsequent generations are indexed Gen 1 through Gen 3.

Three training configurations are evaluated within this unified Cyber Range. The first two configurations serve as baselines, while the third is the proposed MG-CoE training configuration. In Fixed Opponent Training (FOT), the Blue-team agent continues training against the shared Gen 0 Red-team agent, which is fixed and never retrained. This configuration represents the conventional single-adversary baseline and tests whether a Blue-team policy trained against one static attacker can generalise to stronger future Red-team agents. In Simultaneous Co-Evolution (SCE), both Red-team and Blue-team agents continue from the shared Gen 0 baseline and update their policies at every episode, with no agent held fixed. This configuration represents a concurrent self-play baseline and tests whether mutual adaptation alone is sufficient for stable improvement. In Alternating Co-Evolution (ACE), both agents also start from the shared Gen 0 baseline, but Red-team and Blue-team agents train in alternating generations: one agent is held fixed while the other trains against it, and the training agent must improve at least $+2$ % in Red-team kill-chain success rate, or at least $+1.5$ % in Blue-team suppression rate before the generation advances.

All policies are evaluated over $1{,}500$ episodes per seed using two fixed seeds ($\{100, 123\}$), yielding $3{,}000$ total evaluation episodes per reported data point. An exploration rate of $\epsilon_{\text{eval}} = 0.25$ is applied during evaluation to prevent degenerate deterministic policies.

Four metrics are reported throughout this chapter. *Blue-team suppression rate* is the fraction of evaluation episodes in which the Blue-team agent prevents the Red-team agent from reaching the goal host; it serves as the primary measure of defensive quality. *Red-team kill-chain success rate* is the fraction of evaluation episodes in which the Red-team agent completes its objective; it measures offensive effectiveness under each defensive policy. *Unique kill chains* counts the number of distinct multi-stage attack sequences observed across evaluation episodes within a generation and is accumulated cumulatively to quantify Red-team attack diversification. *Kill-chain length distribution* captures the spread of total steps in successful Red-team kill chains; longer chains indicate that the Blue-team agent is closing shallow attack paths and forcing the Red-team agent to penetrate deeper to succeed. Training reward curves, technique frequency heatmaps across MITRE ATT&CK identifiers, and kill-chain progression distributions are used

as supporting diagnostic signals.

6.2 Alternating Co-Evolution vs FOT vs Simultaneous Co-Evolution

The first evaluation compares the three training configurations to determine which produces the most robust Blue-team defense against opponents of varying strength. The central comparison evaluates each configuration against both Red-team Gen 1, representing a low-level adversary, and Red-team Gen 3, representing the stronger co-evolved attacker.

6.2.1 Blue-team Suppression Against Moderate and Strong Red-team Agents

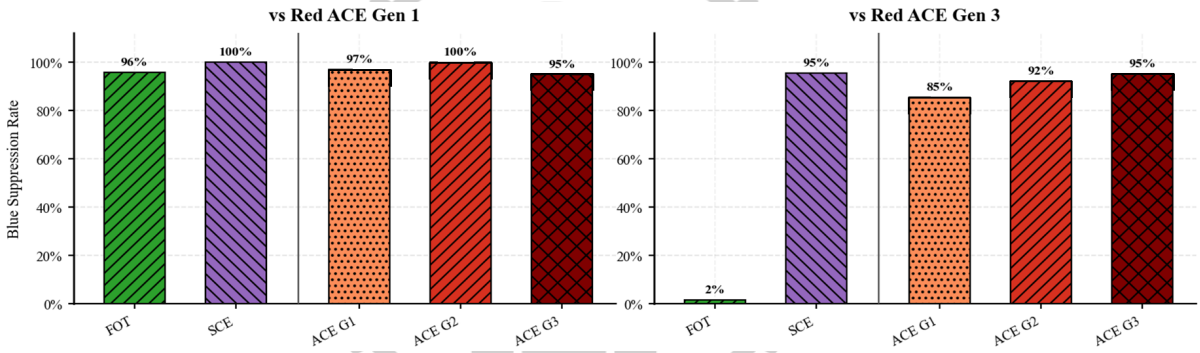

Figure 6.1: Blue-team suppression rate against Red-team Gen 1 and Gen 3.

FOT achieves 96% suppression against Red-team Gen 1, confirming that fixed-opponent training can produce a high-performing policy against a low-level adversary. However, when evaluated against Red-team Gen 3, FOT drops to only 2% suppression. Because the Red-team opponent never adapts during FOT training, the Blue-team agent learns reactive shortcuts that are effective only against Gen 1 strategies and fails to generalise to evolved attack variants. When Red-team Gen 3 employs techniques and kill-chain orderings absent during training, the FOT policy has no learned response and fails in nearly all evaluated episodes.

SCE performs differently. Its best generation achieves 100% suppression against Red-team Gen 1 and 95% against Red-team Gen 3, values that are competitive with the ACE ceiling. However, SCE does not exhibit a monotonic generational improvement trajectory. Because both Red-team and Blue-team agents update simultaneously, the optimisation landscape remains non-

stationary from either agent's perspective. Every Blue-team improvement immediately changes the Red-team policy that the Blue-team agent is implicitly targeting, and vice versa. As a result, performance at any checkpoint reflects a transient equilibrium between two continuously shifting policies, so later checkpoints are not necessarily stronger than earlier ones.

ACE achieves 85% suppression at Gen 1, 92% at Gen 2, and 95% at Gen 3 against Red-team Gen 3, producing a strictly monotonic improvement trajectory with no generation falling below 85% suppression. The alternating promotion structure explains this behavior. By holding one agent fixed while the other trains, each generation provides a stable optimisation target that allows convergence. The promotion threshold then prevents advancement unless measurable improvement is verified, ensuring that each successor policy is stronger than its predecessor.

6.2.2 Training Reward Stability

Training reward dynamics provide additional insight into the stability of each configuration.

Figure 6.2: Moving-average Red-team reward and Blue-team reward across training progress.

The FOT Blue-team reward stabilises near a moving-average reward of approximately +40 throughout training, producing a smooth and well-converged curve. This apparent stability is misleading. Because the Red-team opponent is fixed and weak, the Blue-team agent can achieve high reward by learning a small set of reliable shortcuts without developing a generalisable policy. The smooth convergence therefore reflects the simplicity of optimising against a non-adapting target rather than the quality of the learned defense.

SCE exhibits persistent oscillation throughout training, with neither Red-team nor Blue-team reward converging to a stable plateau. Since both policies evolve simultaneously, every Blue-team improvement destabilises the reward landscape because the Red-team agent it is implicitly

targeting is also changing. This mutual interference makes it difficult to determine whether meaningful policy improvement has actually occurred.

ACE displays a different pattern. Reward curves stabilise within each generation but exhibit sharp disruptions at generational boundaries followed by recovery and re-stabilisation. These disruptions occur whenever a stronger Red-team opponent is introduced, temporarily invalidating portions of the Blue-team agent's prior value estimates. Because the Red-team agent is then held fixed for the remainder of the generation, the Blue-team agent can re-optimise against the new target and recover. The disruption-then-recovery pattern therefore confirms that the Blue-team agent is adapting to progressively harder opponents rather than repeatedly overfitting to the same adversary.

6.2.3 Training Episode Efficiency

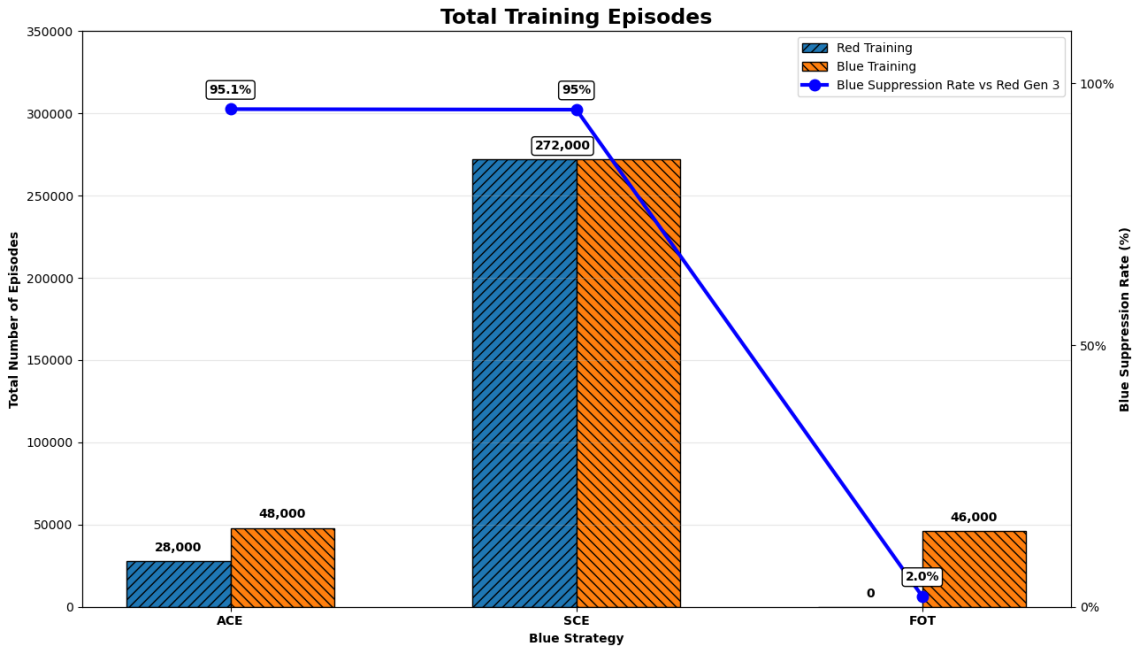

Figure 6.3: Training episode cost.

We further compare training episode cost to examine how much training each configuration requires to reach its reported Blue-team suppression rate. Figure 6.3 compares the total training episodes used by FOT, SCE, and ACE together with their Blue-team suppression rate against Red-team Gen 3.

ACE achieves 95.1% suppression against Red-team Gen 3 using $76{,}000$ training episodes. SCE achieves a similar suppression rate of 95.0%, but requires $272{,}000$ training episodes. There-

fore, ACE reaches comparable defensive performance with approximately $3.6\times$ fewer episodes, or about 172,000 fewer training episodes than SCE. FOT uses fewer episodes than both methods, but its suppression rate drops to only 2% against Red-team Gen 3, showing that low episode cost alone does not indicate robust defense.

This difference is mainly due to the training structure. In ACE, one agent trains against a fixed opponent within each generation, so the learning target is more stable. In SCE, both agents update at the same time, so the opponent policy keeps changing during training. As a result, SCE requires more episodes to reach a similar suppression rate, while ACE provides a more efficient training process.

6.2.4 Kill-Chain Length Behavior

The behavioral differences between configurations are further reflected in the kill-chain length distributions.

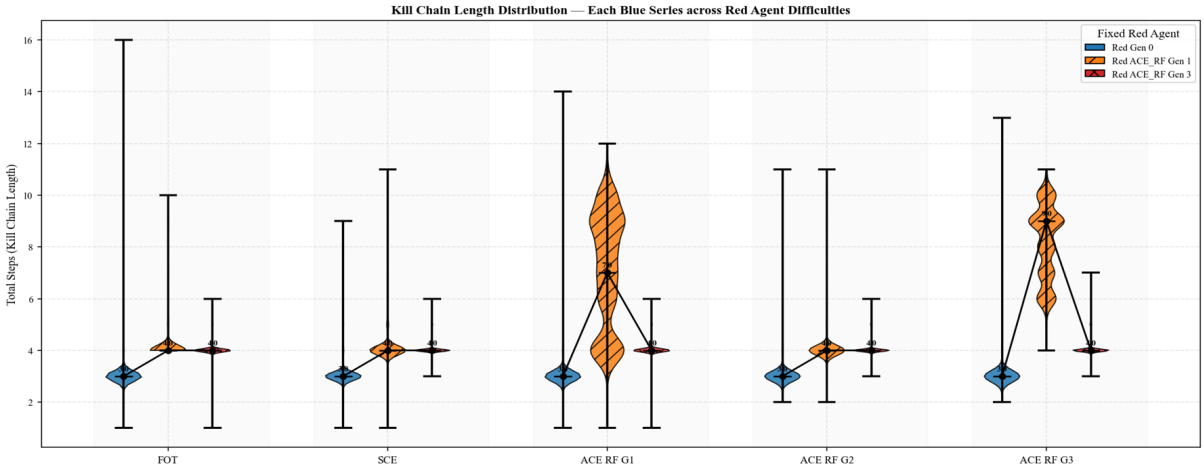

Figure 6.4: Kill-chain length distributions across Blue-team series

Against Red-team Gen 1, FOT and SCE terminate Red-team progress at a median kill-chain length of approximately four steps. ACE Gen 1 exhibits a wider distribution with a median near seven steps, reflecting that the Blue-team agent is still in an early adaptation phase and has not yet developed consistent termination capability. By ACE Gen 2 and Gen 3, the distribution compresses toward a median of four steps and narrows substantially, indicating that mature ACE Blue-team policies consistently intercept Red-team Gen 1 attacks early in the kill chain.

Against Red-team Gen 3, the distribution across all setups compresses near four steps. This

does not indicate equal defensive effectiveness. Red-team Gen 3 has evolved highly efficient attack paths that achieve the goal with minimal exposure, so short successful chains reflect offensive sophistication rather than Blue-team success. FOT's 2% suppression rate confirms that compressed chains in FOT result from Red-team attacks executing faster than the Blue-team agent can respond. ACE Gen 3, by contrast, achieves 95% suppression against the same Red-team Gen 3 opponent despite the compressed chains, meaning that only a small number of highly optimised attacks remain successful against a mature Blue-team defense.

6.3 ACE Generational Dynamics

Having established that ACE produces the most robust and consistent defensive performance, the final evaluation examines the internal generational dynamics of ACE.

6.3.1 Blue-team Suppression Across Generations

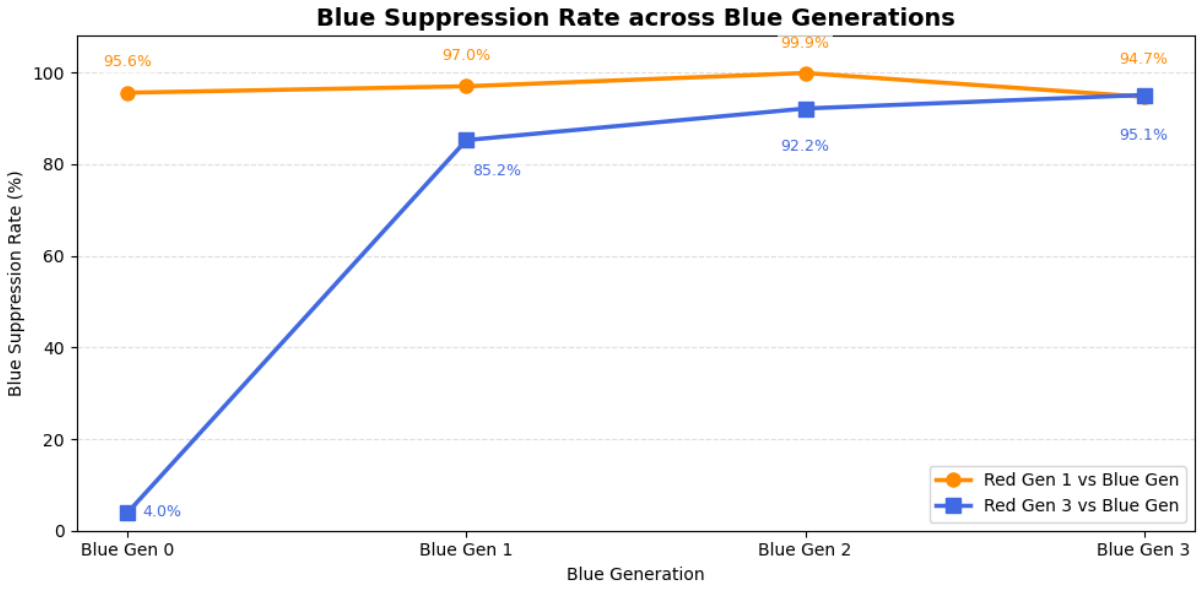

Figure 6.5: ACE Blue-team suppression against Red-team Gen 1 and Red-team Gen 3.

Figure 6.5 shows Blue-team suppression across generations evaluated against both Red-team Gen 1 and Red-team Gen 3. Blue-team suppression against Red-team Gen 3 rises from 4.0% at Gen 0 to 85.2% at Gen 1, 92.2% at Gen 2, and 95.1% at Gen 3, representing a total gain of +91.1 percentage points. The largest improvement occurs between Gen 0 and Gen 1, where the untrained baseline policy is replaced by one explicitly trained against Red-team Gen 1. Suppres-

sion against Red-team Gen 1 remains at or above 95.6% across all generations, confirming that later Blue-team generations retain effectiveness against weaker adversaries while simultaneously learning to suppress stronger ones. The strictly monotonic progression against Red-team Gen 3 demonstrates that each generational training cycle produces genuine defensive improvement against a progressively evolving attacker.

6.3.2 Red-team Kill-Chain Diversity

While the Blue-team agent improves across generations, the Red-team agent simultaneously continues to diversify its attack repertoire.

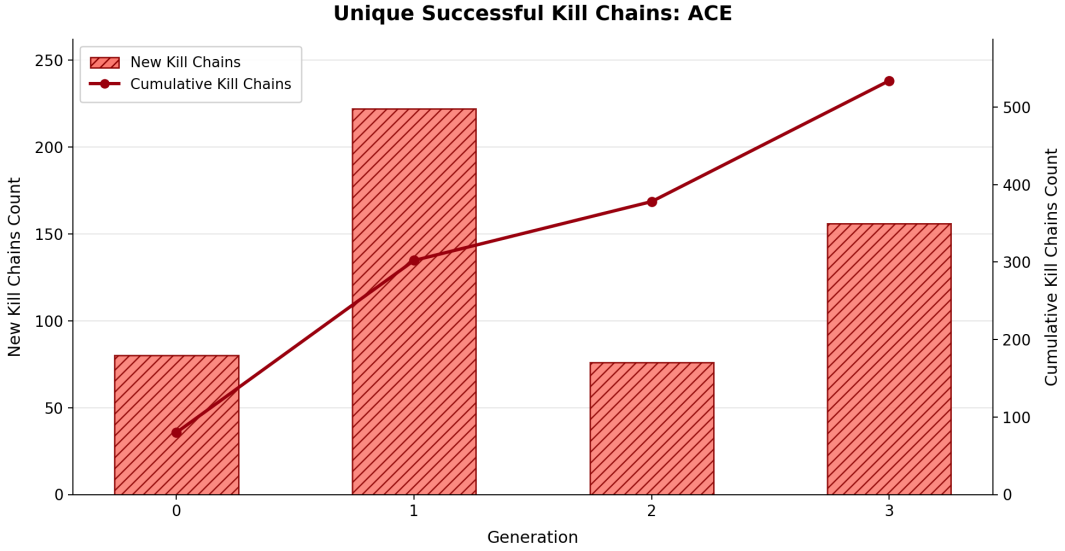

Figure 6.6: New and cumulative unique successful kill chains across Red-team generations.

Gen 0 contributes 80 baseline kill chains. Gen 1 adds 222 new chains, reflecting the large unexplored attack surface available against the untrained Blue-team baseline. Gen 2 adds 75 additional chains, while Gen 3 contributes another 156. The cumulative total increases monotonically from 80 to approximately 533 chains by Gen 3, with each generation introducing attack paths not observed in previous generations. This continuous diversification confirms that the Red-team agent is not simply replaying earlier strategies but actively discovering new attack opportunities in response to the Blue-team agent's improving defense.

6.3.3 Technique Rotation Across Generations

The same adaptation process is visible at the level of individual MITRE ATT&CK techniques.

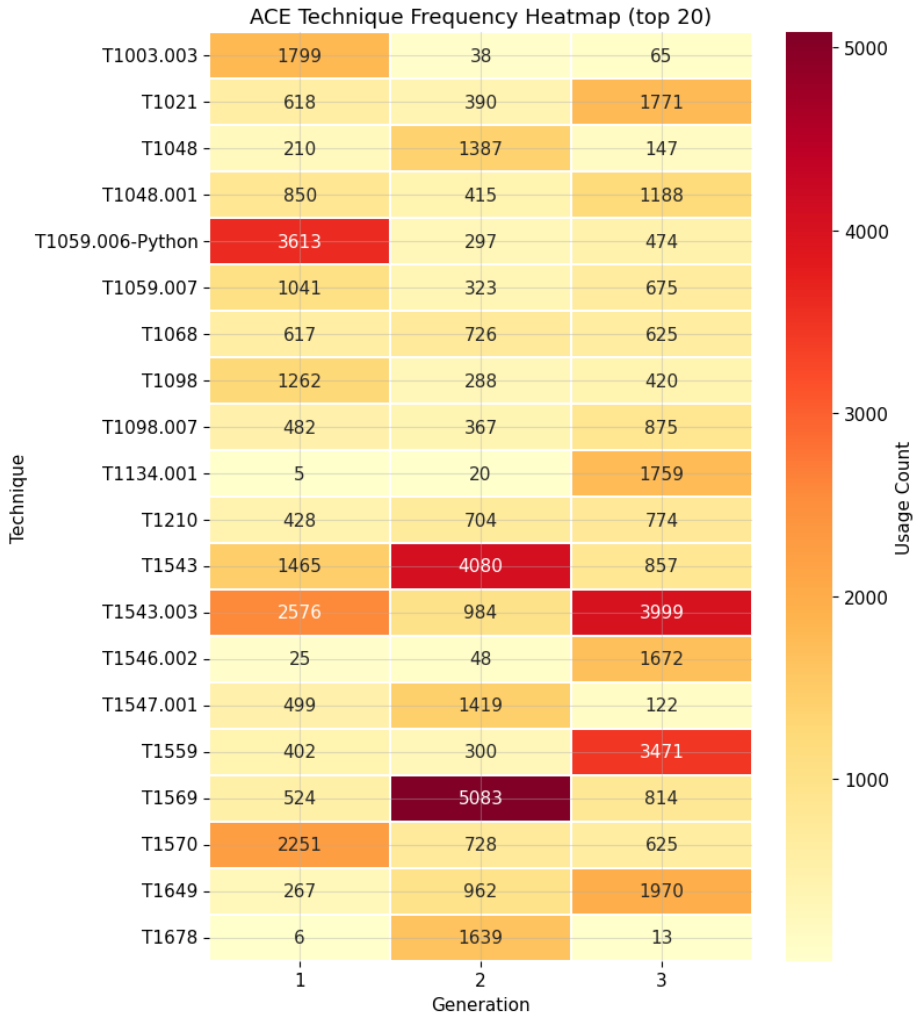

Figure 6.7: ACE technique frequency heatmap across generations.

In Gen 1, T1059.006 (Python execution) dominates with 3,613 usages. In Gen 2, T1569 (System Services) rises sharply to 5,083 usages while T1059.006 drops to only 297. In Gen 3, T1543.003 emerges as the dominant technique with 3,999 usages, while T1569 declines substantially. Additional techniques exhibit similar rotational behavior: T1134.001 increases from only 5 usages in Gen 1 to 1,759 in Gen 3, while T1559 rises from 402 to 3,471 over the same interval.

This rotation emerges directly from the MG-CoE feedback loop. As the Blue-team agent improves detection and suppression for frequently used techniques, their expected utility within

the Red-team policy decreases. Red-team exploration consequently shifts toward alternative techniques that the Blue-team agent has not yet learned to counter effectively. The dominant attack strategy therefore rotates across generations rather than converging to a fixed policy.

6.3.4 Kill-Chain Depth Analysis

The section examines how the depth of successful Red-team attacks changes across generations, as shown in Figure 6.8.

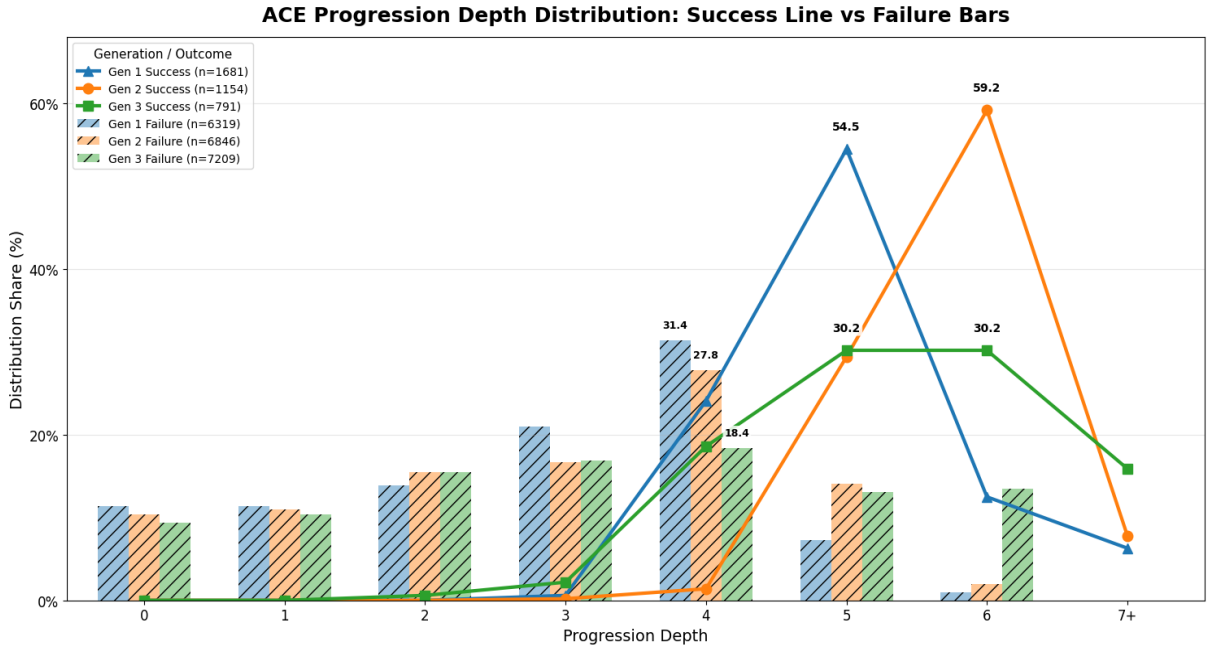

Figure 6.8: ACE kill-chain progression depth distribution.

In Gen 1, the Red-team agent achieves its highest success frequency at progression depth 5, with 1,681 successful episodes and 6,319 failures. In Gen 2, the success peak shifts toward depth 6, while failure counts increase to 6,846. By Gen 3, successful attacks spread across depths 6 and 7, while failures increase further to 7,209 episodes.

Failure episodes remain concentrated at shallow depths across all generations, confirming that the Blue-team agent consistently blocks most attacks early in the kill chain. At the same time, the depth required for successful attacks progressively increases. A mature Blue-team defense therefore does not merely suppress a larger fraction of the same attacks; it systematically eliminates shallow attack paths and forces the Red-team agent to invest more stages to achieve success. The deeper successful chains observed in Gen 3 represent the residual frontier of Red-team capability against a highly adapted Blue-team defense.

6.3.5 Cross-Generation Generalization

A complementary question is whether this improvement generalises across opponents from different generations.

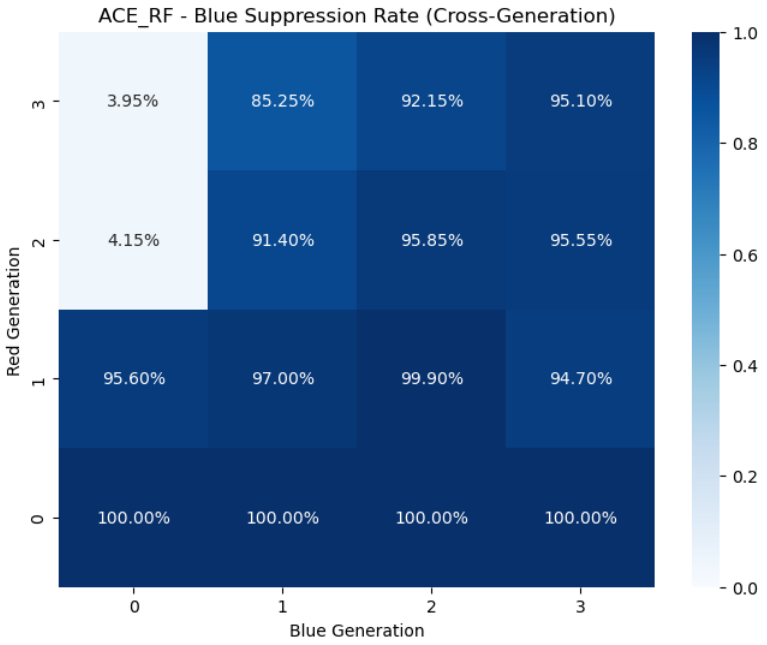

Figure 6.9: ACE cross-generation Blue-team suppression rate heatmap.

The cross-generation suppression matrix reveals two consistent structural patterns. Reading any row from left to right holding Red-team generation constant while advancing Blue-team generation suppression generally increases. For example, against Red-team Gen 2, Blue-team Gen 0 suppresses only 4.15% of attacks, whereas Blue-team Gen 1 suppresses 91.40% and Blue-team Gen 2 suppresses 95.85%. This progression confirms that later Blue-team generations accumulate transferable defensive knowledge rather than overfitting to a single matched-generation opponent.

Reading any column from bottom to top holding Blue-team generation constant while increasing Red-team strength suppression decreases as Red-team capability becomes more sophisticated. The strongest contrast occurs between Blue-team Gen 0 against Red-team Gen 3, which achieves only 3.95% suppression, and Blue-team Gen 3 against Red-team Gen 3, which reaches 95.10%. Together, these row-wise and column-wise trends form the empirical signature of a bidirectional arms race in which each agent's progression depends on and builds upon the evolution of the opposing agent.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis addressed a key limitation in automated cyber defense training: the lack of a structured and verifiable mechanism for ensuring that both attacker and defender agents genuinely improve across training. Fixed-opponent training can lead to overfitting against a static adversary, while simultaneous co-evolution introduces non-stationarity that makes improvement difficult to verify.

To address this problem, this thesis proposed the Multi-generational Co-Evolution (MG-CoE) framework. MG-CoE organises adversarial training into alternating generations, where one agent is held fixed while the opposing agent trains against it. A candidate agent is promoted only when it satisfies a quantitative improvement threshold. This design converts unstable joint optimisation into a sequence of single-agent learning problems against stable but progressively stronger opponents.

The framework integrates three main components. First, a reconnaissance-driven base state provides a consistent starting environment for each episode. Second, the Cyber Range and Range Controller ground Red-team actions in MITRE ATT&CK technique schemas, including preconditions, state transitions, observable evidence, and detectability. Third, the Red-team and Blue-team agents are implemented using DDQN, where the Red-team agent learns parameterised kill-chain execution and the Blue-team agent learns detection and response.

The evaluation compared Alternating Co-Evolution (ACE) with Fixed Opponent Training (FOT) and Simultaneous Co-Evolution (SCE). The results showed that ACE produces the most stable and verifiable improvement. Against the strongest Red-team attacker, Blue-team suppression increased monotonically from 4.0% at Generation 0 to 85.2%, 92.2%, and 95.1% across Generations 1 to 3. In contrast, FOT dropped to only 2% suppression against the same oppo-

nent, while SCE did not provide a monotonic improvement trajectory. Cross-generation results also showed that later Blue-team agents generalise better across Red-team generations, while technique-frequency analysis showed that the Red-team agent continues to rotate strategies in response to stronger Blue-team defenses.

Overall, the results demonstrate that MG-CoE provides a practical and measurable approach to automated cyber defense training. By combining alternating training, promotion-based verification, and Cyber Range-based interaction, the framework produces progressively stronger agents while maintaining a clear evaluation process for each generation.

7.2 Future Work

Future work can extend MG-CoE in three directions. First, the technique catalogue and Cyber Range environment can be expanded to reduce the realism gap of MITRE-based simulation. The current implementation uses a fixed MITRE ATT&CK catalogue and a flat five-host environment. Future work could add more tactics, sub-techniques, cross-platform variants, multi-subnet networks, and heterogeneous operating systems. It could also incorporate CVE-based attack data while preserving MITRE mapping, linking specific CVEs to ATT&CK techniques with concrete commands, parameters, payloads, and Metasploit-like procedures. This would make the simulated actions closer to real exploitation behaviour.

Second, the framework can be connected to real execution and raw telemetry. The current implementation simulates attack actions and generates telemetry from technique schemas. A future version could execute Red-team actions in an isolated Cyber Range using tools such as Metasploit or Atomic Red Team, and collect raw logs from SIEM, IDS, and endpoint monitoring tools. This would expose the Blue-team component to realistic noise, ambiguity, and help assess whether detection policies remain robust beyond schema-generated telemetry.

Third, future work can explore online LLM support and multi-agent scenarios. The current LLM usage is offline and mainly supports schema generation and base-state preparation. Online LLM support could help interpret raw logs, explain defensive decisions, or suggest new Red-team paths during interaction. Extending MG-CoE to multi-agent Red-team and Blue-team settings would better reflect coordinated security operations.

References

[1] Valea, Ovidiu and Oprisa, Ciprian, "Towards Pentesting Automation Using the Metasploit Framework," in *2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing (ICCP)*, 2020, pp. 171–178.

[2] Red Canary, "Atomic Red Team," 2024. [Online]. Available: https://atomicredteam.io/

[3] H. P. T. Nguyen, K. Hasegawa, K. Fukushima, and R. Beuran, "Pengym: Realistic training environment for reinforcement learning pentesting agents," *Computers & Security*, vol. 148, p. 104140, 2025.

[4] S. R. Castro, R. Campbell, N. Lau, O. Villalobos, J. Duan, and A. A. Cardenas, "Large language models are autonomous cyber defenders," in *2025 IEEE Conference on Artificial Intelligence (CAI)*. IEEE, May 2025, p. 1125—1132. [Online]. Available: http://dx.doi.org/10.1109/CAI64502.2025.00195

[5] H. Emerson, L. Bates, C. Hicks, and V. Mavroudis, "Cyborg++: An enhanced gym for the development of autonomous cyber agents," Oct 2024.

[6] S. Oesch, A. Chaulagain, P. Austria, B. Weber, A. Sadovnik, C. Watson, M. Dixson, and B. Roberson, "Towards a high fidelity training environment for autonomous cyber defense agents," in *Workshop on Cyber Security Experimentation and Test (CSET)*, 2024, pp. 1–9.

[7] L. Li, J.-P. S. El Rami, A. Taylor, J. H. Rao, and T. Kunz, "Cygil: Enabling a network ai gym for autonomous cyber agents," in *International Conference on Computational Science and Computational Intelligence (CSCI)*, 2022, pp. 172–179.

[8] T. Kunz, C. Fisher, J. L. Novara-Gsell, C. Nguyen, and L. Li, "A multiagent cyberbattlesim for rl cyber operation agents," 2023. [Online]. Available: https://arxiv.org/abs/2304.11052

[9] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016, pp. 2094–2100.

[10] A. Nelson, S. Rekhi, M. Souppaya, and K. Scarfone, “Incident response recommendations and considerations for cybersecurity risk management: A csf 2.0 community profile,” National Institute of Standards and Technology, Special Publication 800-61 Rev. 3, Apr. 2025. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-61r3

[11] Strom, Blake and others, “MITRE ATT&CK: Design and Philosophy,” in *Technical report*. The MITRE Corporation, 2018. [Online]. Available: https://www.mitre.org/sites/default/files/2021-11/prs-19-01075-28-mitre-attack-design-and-philosophy.pdf

[12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[13] Wiering, Marco A and Van Otterlo, Martijn, “Reinforcement learning,” *Adaptation, learning, and optimization*, vol. 12, no. 3, p. 729, 2012.

[14] W. Masson, P. Ranchod, and G. Konidaris, “Reinforcement learning with parameterized actions,” 2015.

[15] R. Kuppa, S. Poudel, C. E. Hughes, M. F. Alzaben, and P. Tague, “Adversarial reinforcement learning for ransomware attack path generation and defense evaluation,” Apr 2024.

[16] Motlagh, Farzad and Hajizadeh, Mehrdad and Majd, Mehryar and Najafi, Pejman and Cheng, Feng and Meinel, Christoph, “Large Language Models in Cybersecurity: State-of-the-Art,” 2024. [Online]. Available: https://arxiv.org/abs/2402.00891

[17] Microsoft, “CyberBattleSim,” https://github.com/microsoft/CyberBattleSim, 2021, gitHub repository.

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.

[19] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with numpy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[20] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[21] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[22] Broadcom, "Vmware workstation pro documentation," https://techdocs.broadcom.com/us/en/vmware-cis/desktop-hypervisors/workstation-pro/17-0.html, 2026, accessed: 2026-05-27.

[23] G. Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Insecure.Com LLC, 2008. [Online]. Available: https://nmap.org/book/

[24] Wazuh, "Wazuh documentation," https://documentation.wazuh.com/current/index.html, 2026, accessed: 2026-05-27.

[25] Open Information Security Foundation, "Suricata user guide," https://docs.suricata.io/, 2026, accessed: 2026-05-27.

[26] OpenAI, "OpenAI API Reference," https://developers.openai.com/api/reference/overview/, 2026, accessed: 2026-05-25.